

Neural Networks

©2016 Fjodor van Veen - asimovinstitute.org

-  Backfed Input Cell
-  Input Cell
-  Noisy Input Cell
-  Hidden Cell
-  Probabilistic Hidden Cell
-  Spiking Hidden Cell
-  Output Cell
-  Match Input Output Cell
-  Recurrent Cell
-  Memory Cell
-  Different Memory Cell
-  Kernel
-  Convolution or Pool

Perceptron (P)



Feed Forward (FF)



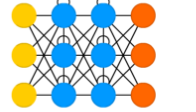
Radial Basis Network (RBF)



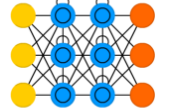
Deep Feed Forward (DFF)



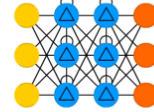
Recurrent Neural Network (RNN)



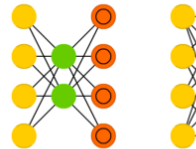
Long / Short Term Memory (LSTM)



Gated Recurrent Unit (GRU)



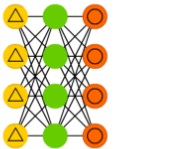
Auto Encoder (AE)



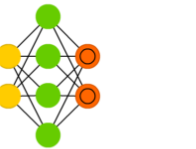
Variational AE (VAE)



Denoising AE (DAE)



Sparse AE (SAE)



Markov Chain (MC)



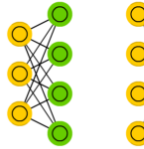
Hopfield Network (HN)



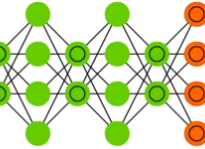
Boltzmann Machine (BM)



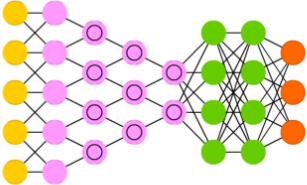
Restricted BM (RBM)



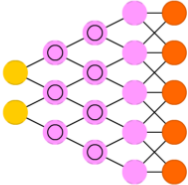
Deep Belief Network (DBN)



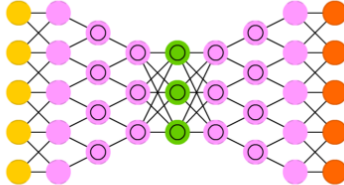
Deep Convolutional Network (DCN)



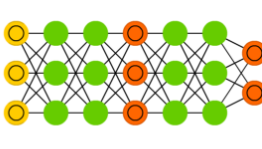
Deconvolutional Network (DN)



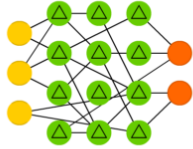
Deep Convolutional Inverse Graphics Network (DCIGN)



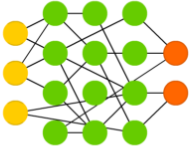
Generative Adversarial Network (GAN)



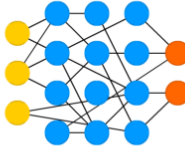
Liquid State Machine (LSM)



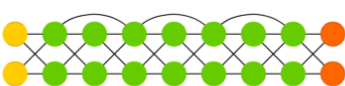
Extreme Learning Machine (ELM)



Echo State Network (ESN)



Deep Residual Network (DRN)



Kohonen Network (KN)



Support Vector Machine (SVM)



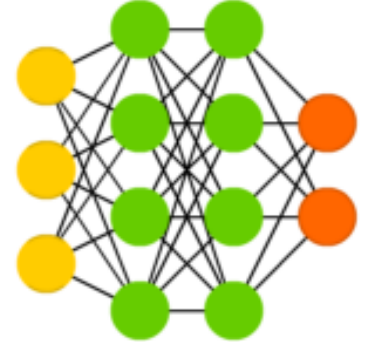
Neural Turing Machine (NTM)



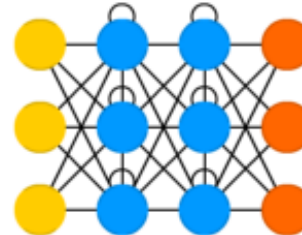
Perceptron (P)



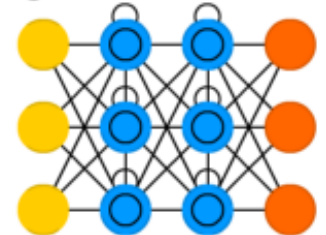
Deep Feed Forward (DFF)



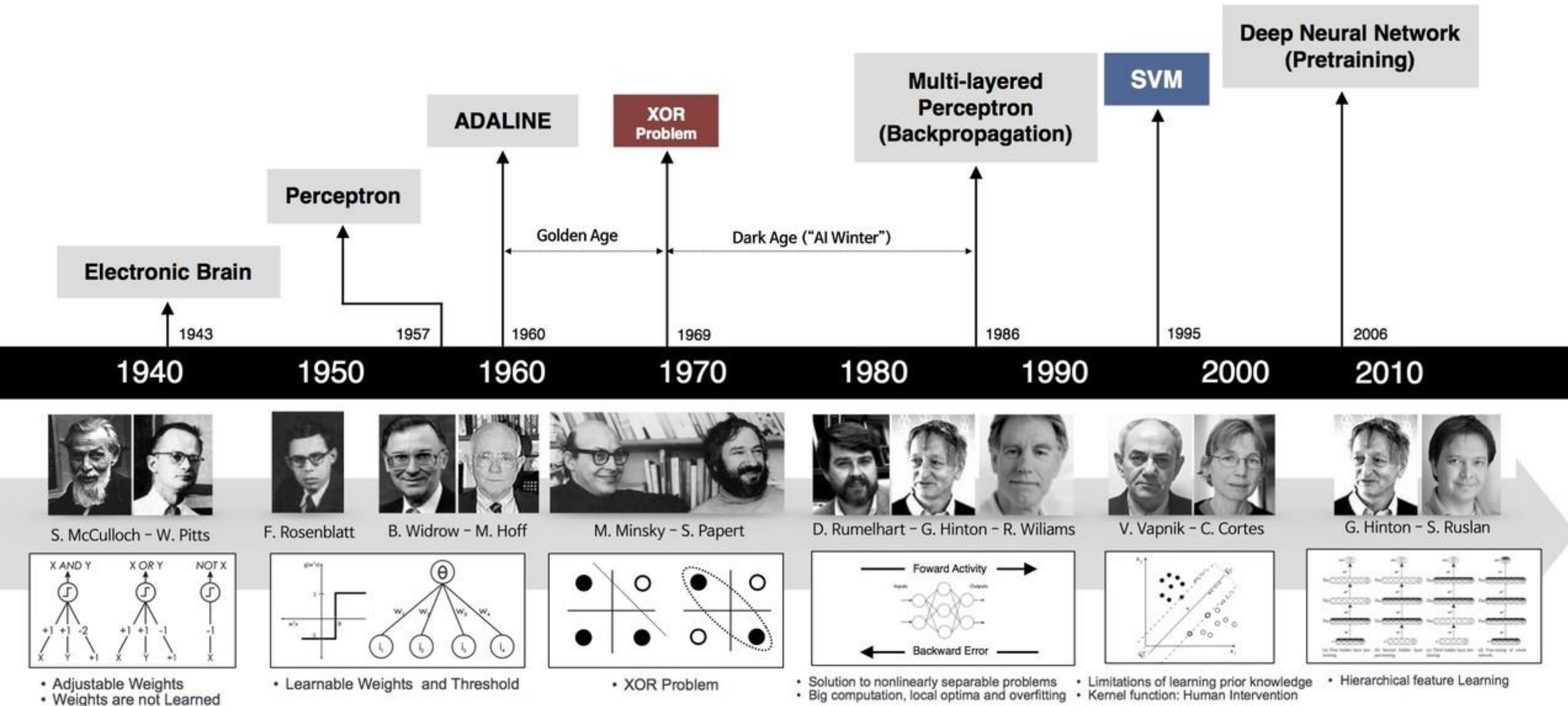
Recurrent Neural Network (RNN)

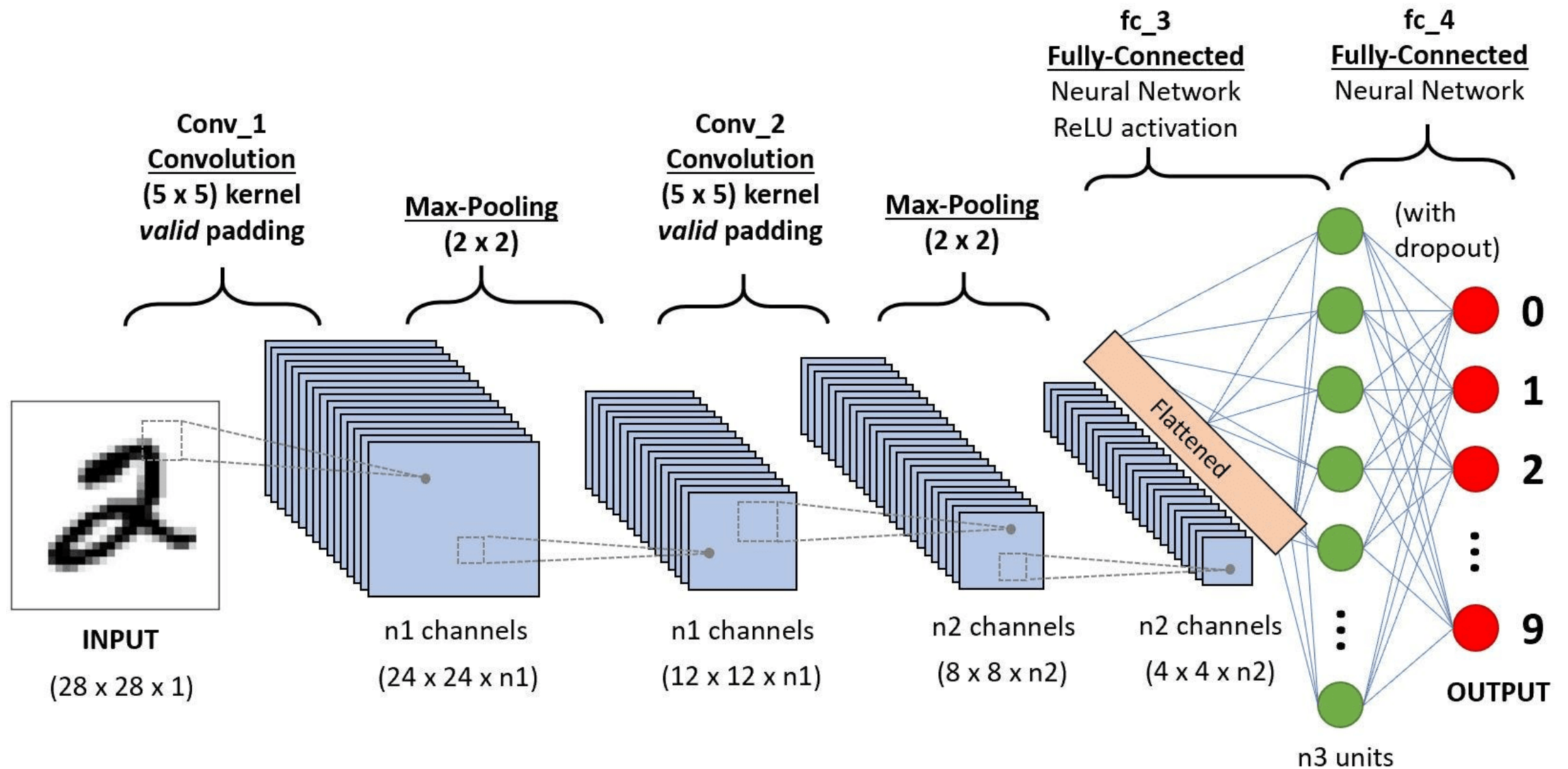


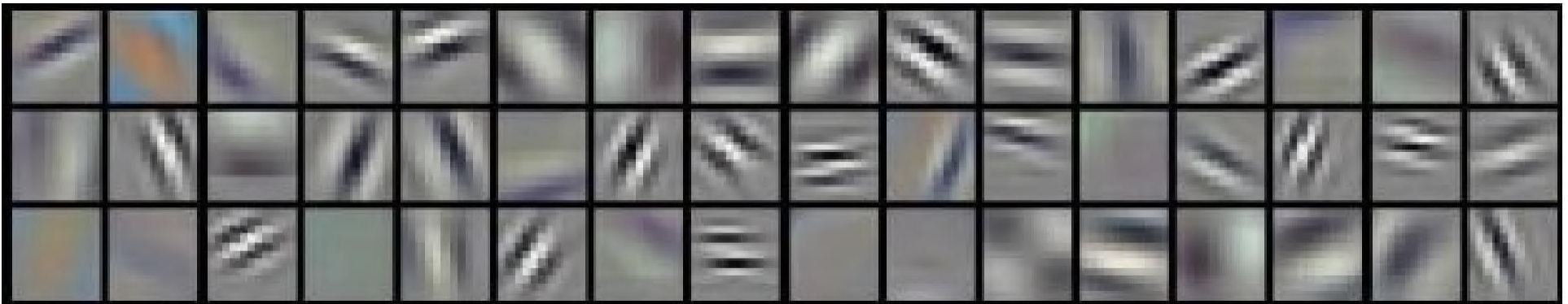
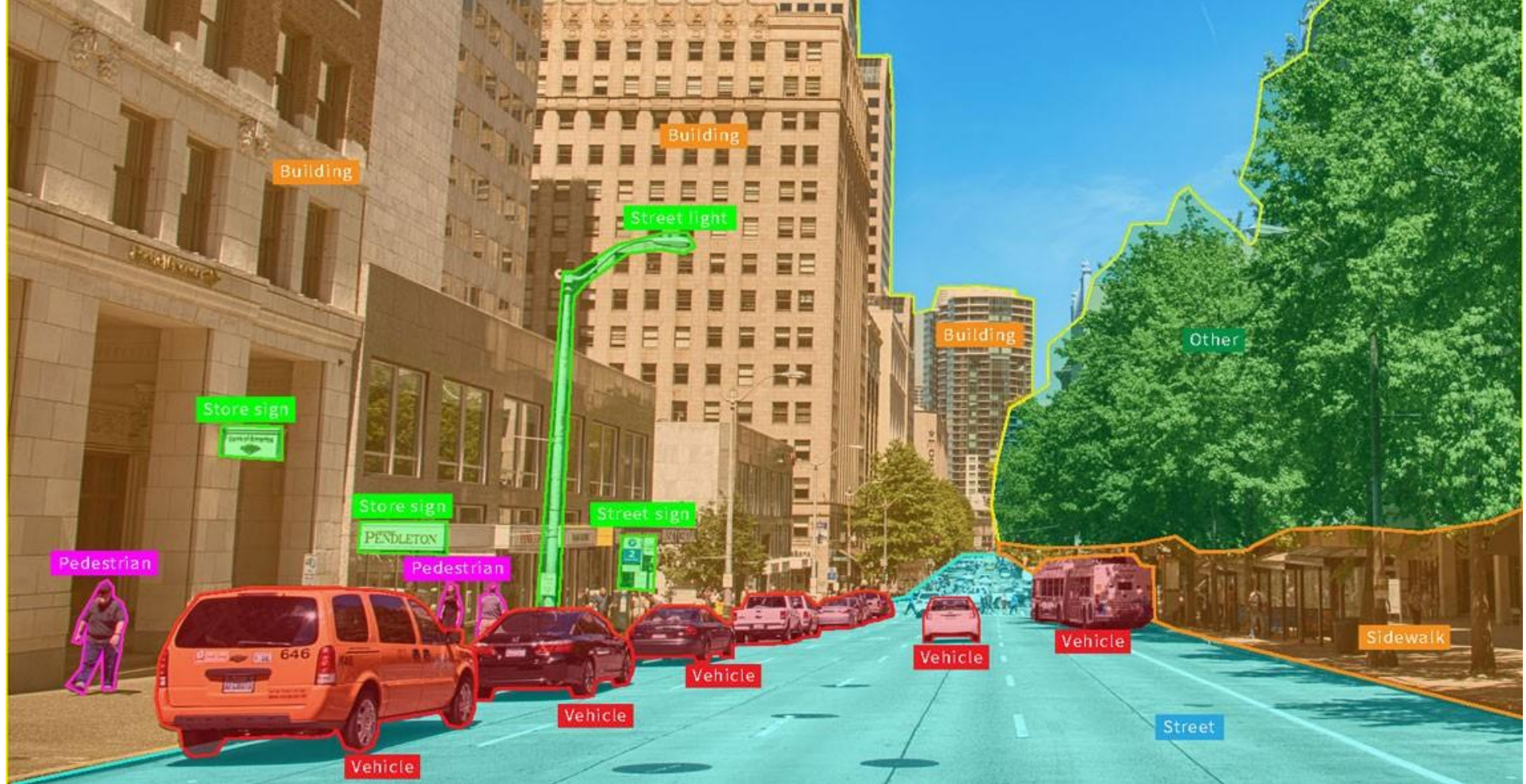
Long / Short Term Memory (LSTM)



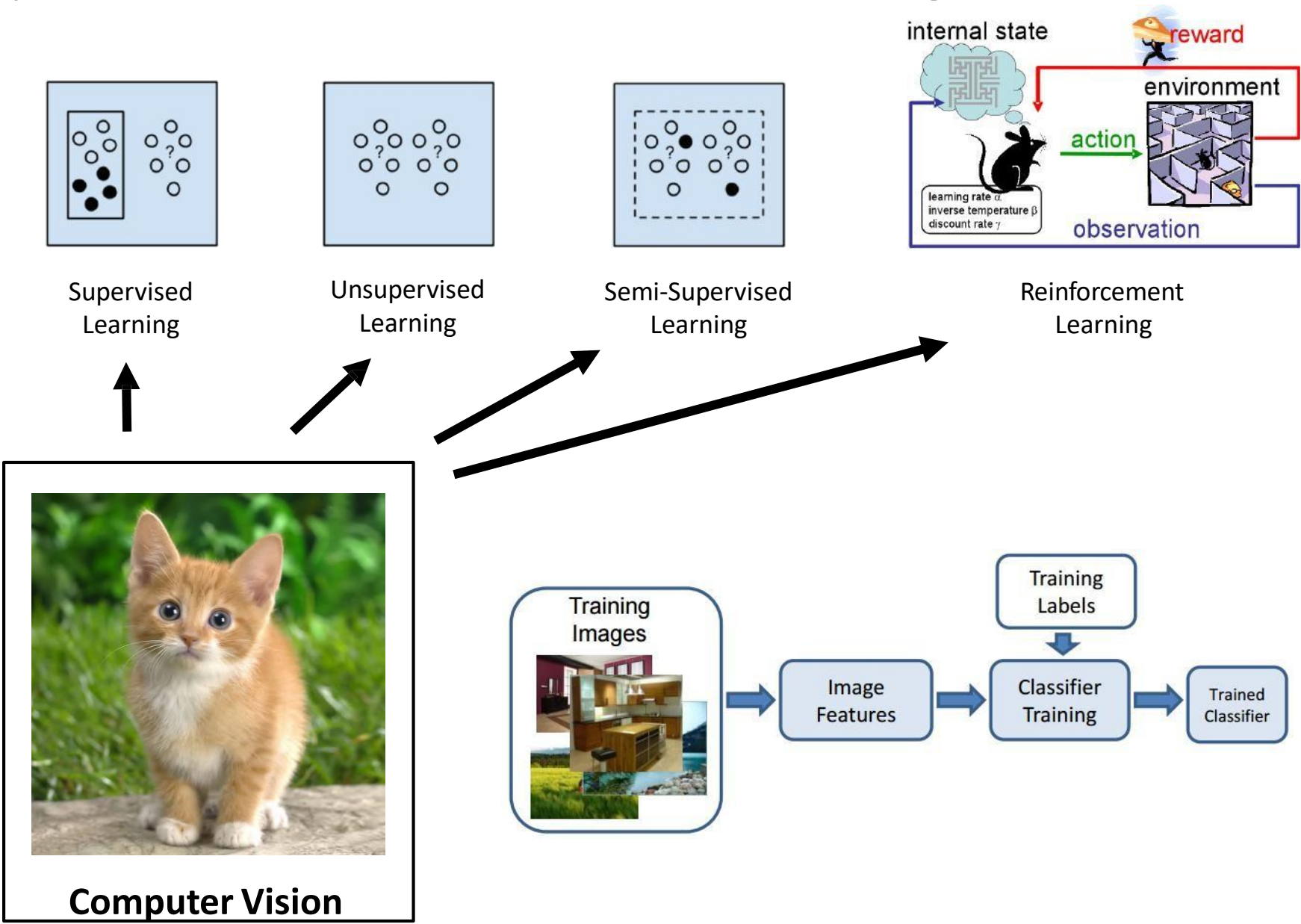
CNN Time line



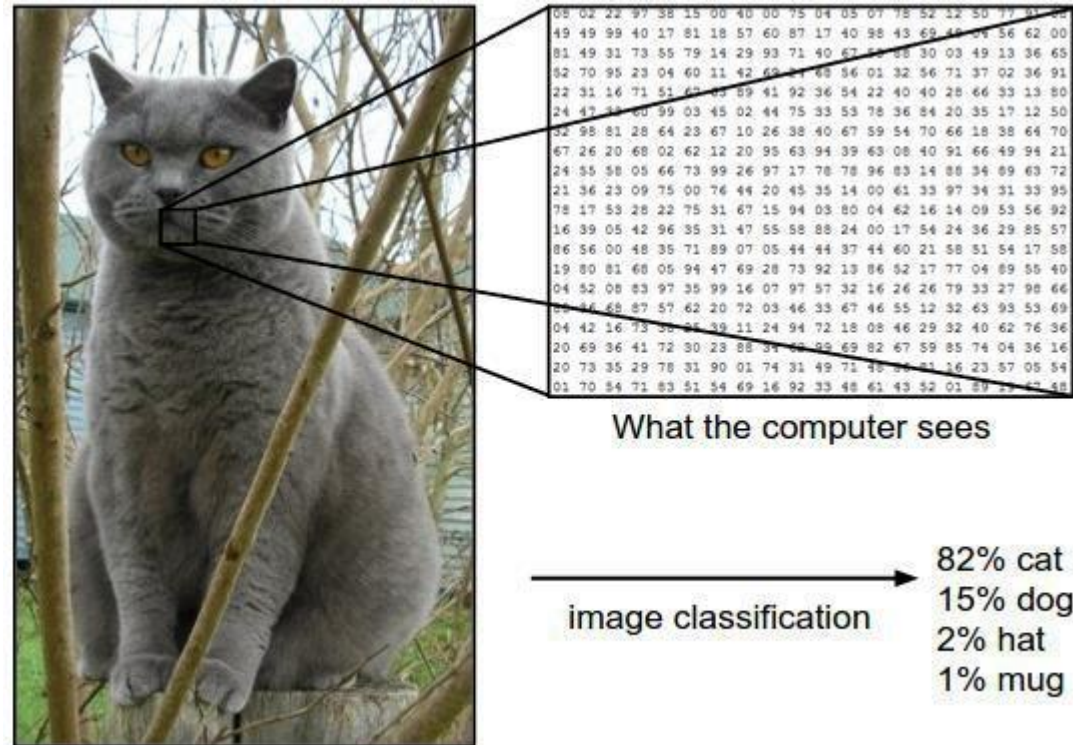




Computer Vision is Machine Learning

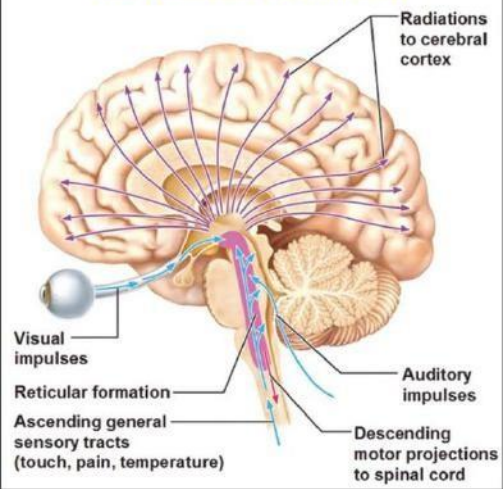


Images are Numbers



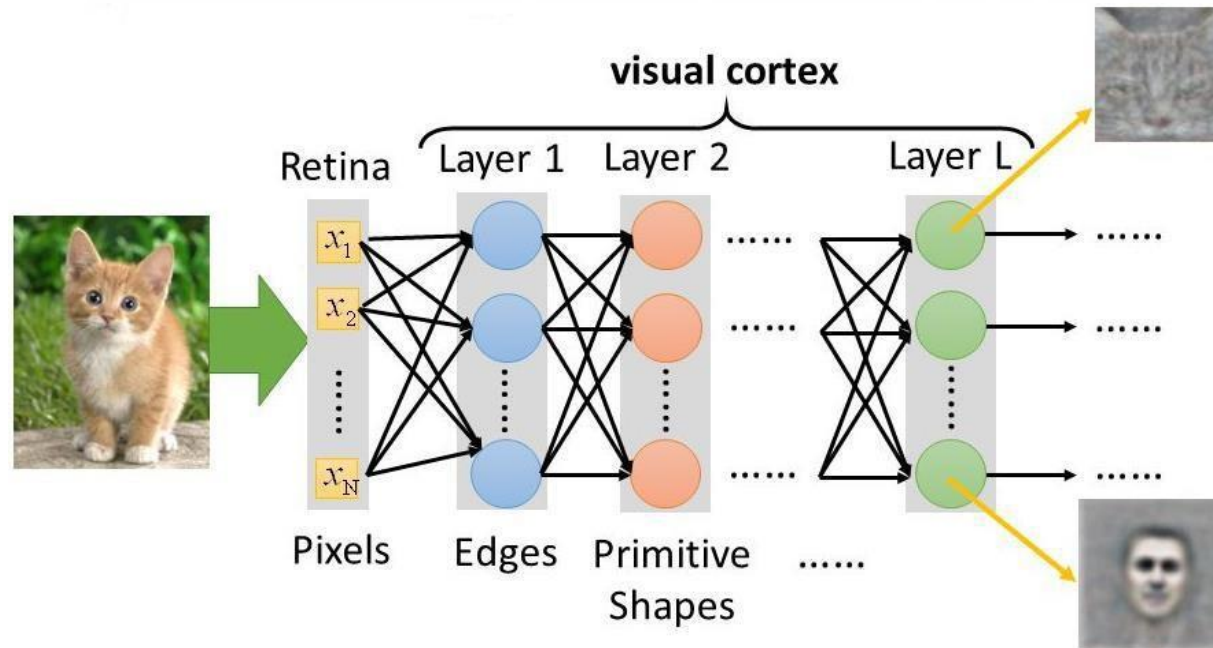
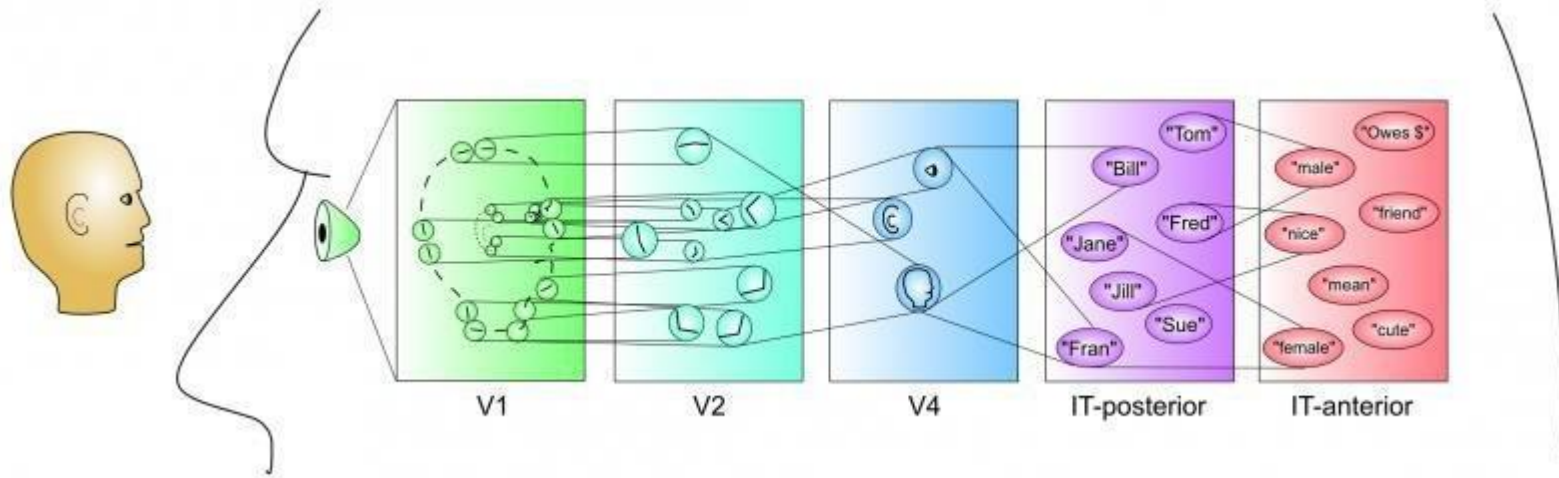
- **Regression:** The output variable takes continuous values
- **Classification:** The output variable takes class labels
 - Underneath it may still produce continuous values such as probability of belonging to a particular class.

The Reticular Formation



Human Vision

Its structure is instructive and inspiring!



Computer Vision is Hard

Viewpoint variation



Scale variation



Deformation



Occlusion



Illumination conditions



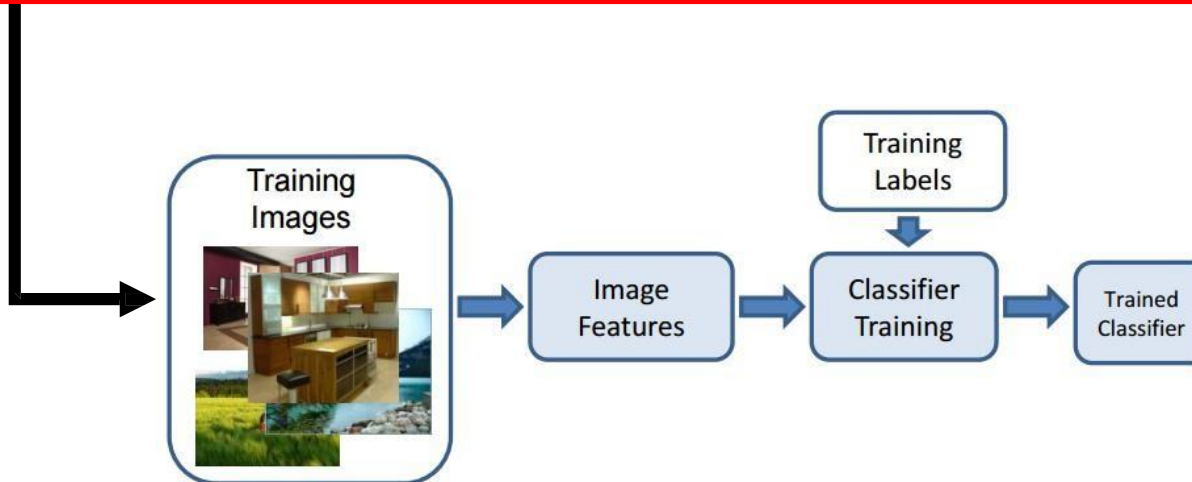
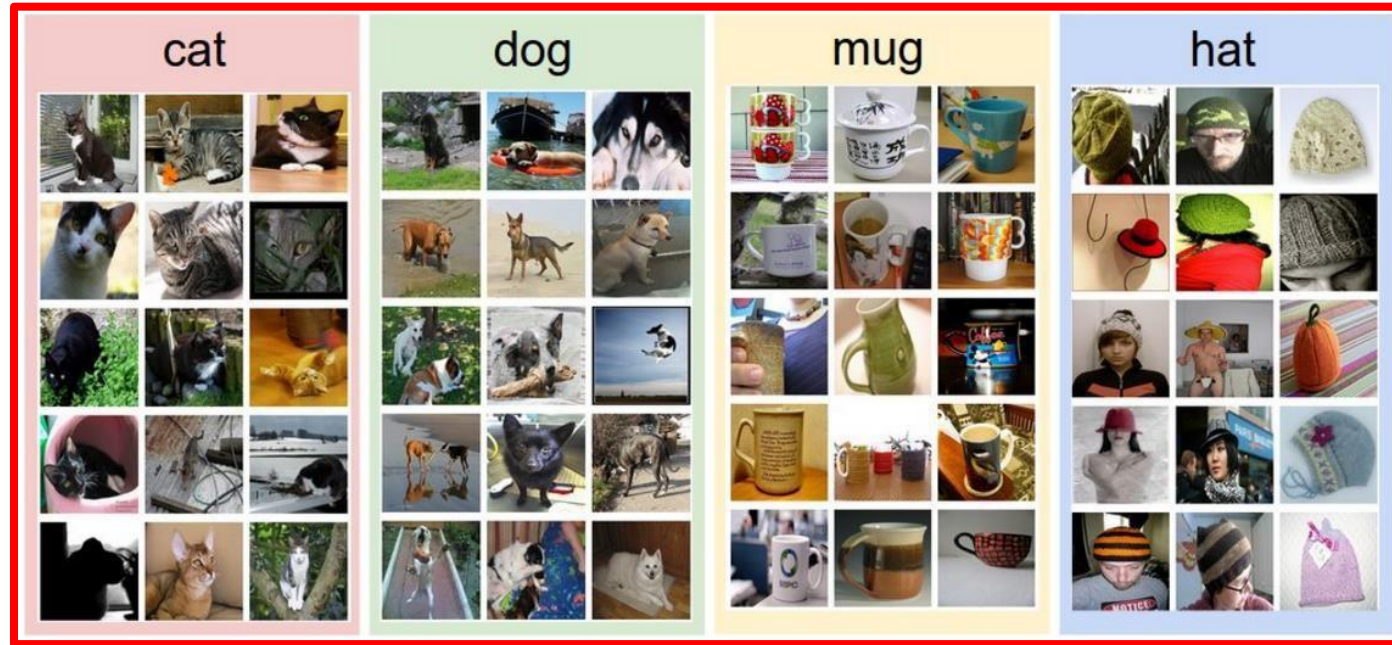
Background clutter



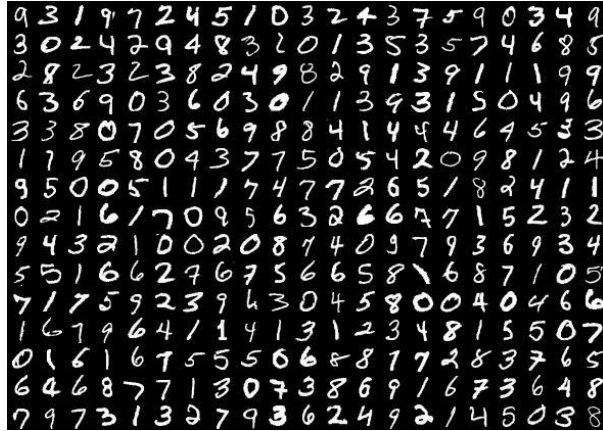
Intra-class variation



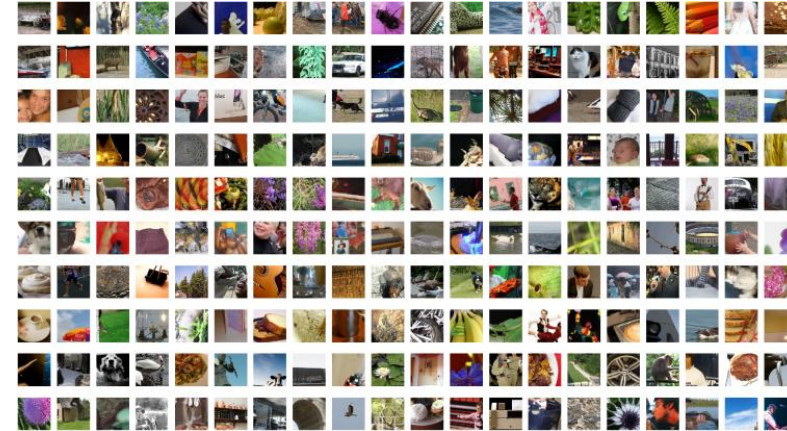
Image Classification Pipeline



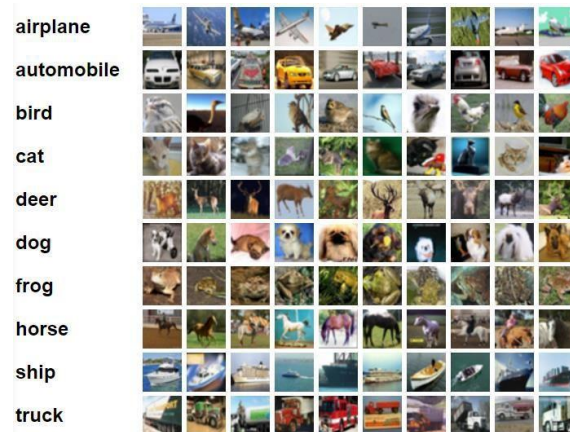
Famous Computer Vision Datasets



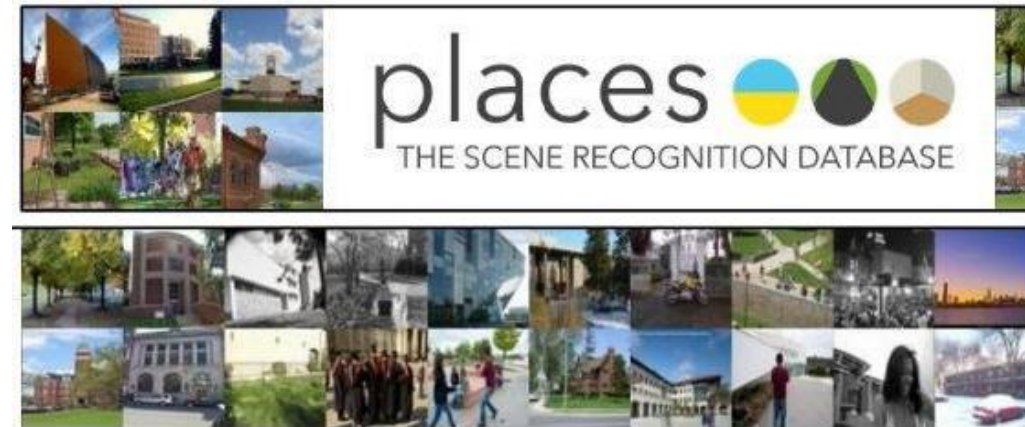
MNIST: handwritten digits



ImageNet: WordNet hierarchy

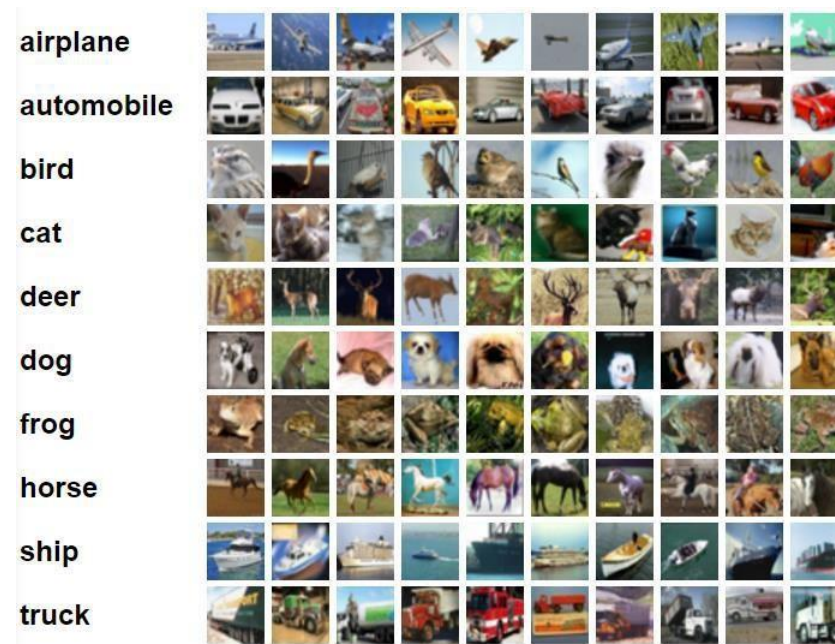


CIFAR-10(0): tiny images



Places: natural scenes

Let's Build an Image Classifier for CIFAR-10



$$\begin{array}{|c|c|c|c|} \hline \text{test image} & & & \\ \hline 56 & 32 & 10 & 18 \\ \hline 90 & 23 & 128 & 133 \\ \hline 24 & 26 & 178 & 200 \\ \hline 2 & 0 & 255 & 220 \\ \hline \end{array} - \begin{array}{|c|c|c|c|} \hline \text{training image} & & & \\ \hline 10 & 20 & 24 & 17 \\ \hline 8 & 10 & 89 & 100 \\ \hline 12 & 16 & 178 & 170 \\ \hline 4 & 32 & 233 & 112 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline \text{pixel-wise absolute value differences} & & & \\ \hline 46 & 12 & 14 & 1 \\ \hline 82 & 13 & 39 & 33 \\ \hline 12 & 10 & 0 & 30 \\ \hline 2 & 32 & 22 & 108 \\ \hline \end{array} \rightarrow 456$$

Let's Build an Image Classifier for CIFAR-10

$$\begin{array}{|c|c|c|c|} \hline \text{test image} & & & \\ \hline 56 & 32 & 10 & 18 \\ \hline 90 & 23 & 128 & 133 \\ \hline 24 & 26 & 178 & 200 \\ \hline 2 & 0 & 255 & 220 \\ \hline \end{array} - \begin{array}{|c|c|c|c|} \hline \text{training image} & & & \\ \hline 10 & 20 & 24 & 17 \\ \hline 8 & 10 & 89 & 100 \\ \hline 12 & 16 & 178 & 170 \\ \hline 4 & 32 & 233 & 112 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline \text{pixel-wise absolute value differences} & & & \\ \hline 46 & 12 & 14 & 1 \\ \hline 82 & 13 & 39 & 33 \\ \hline 12 & 10 & 0 & 30 \\ \hline 2 & 32 & 22 & 108 \\ \hline \end{array} \rightarrow 456$$



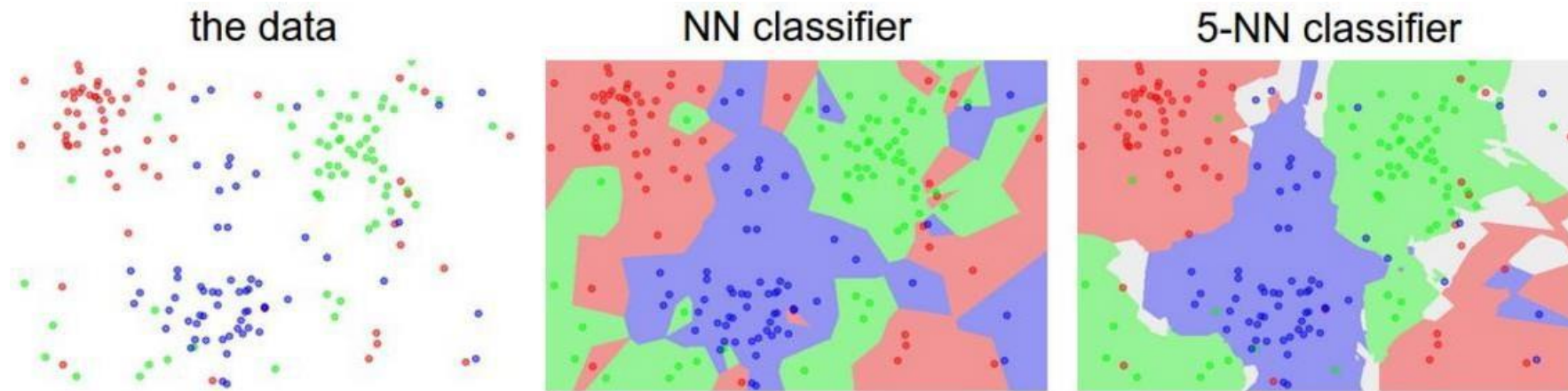
Accuracy

Random: **10%**

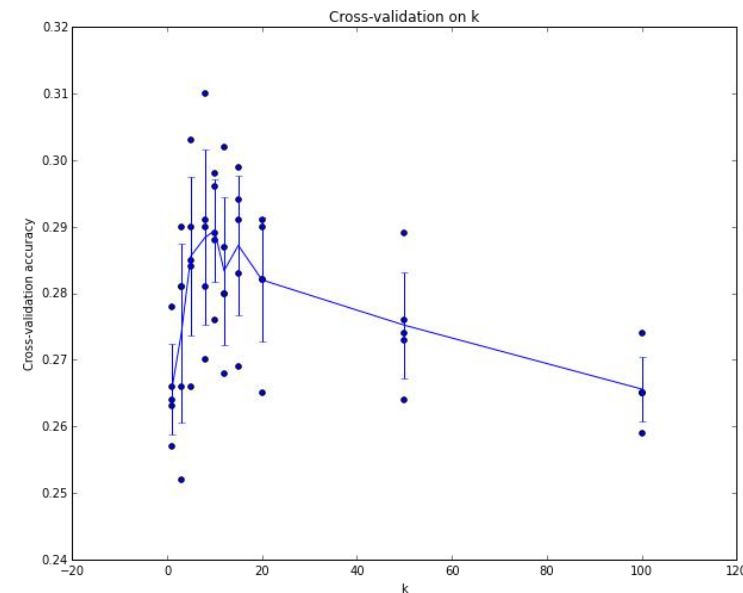
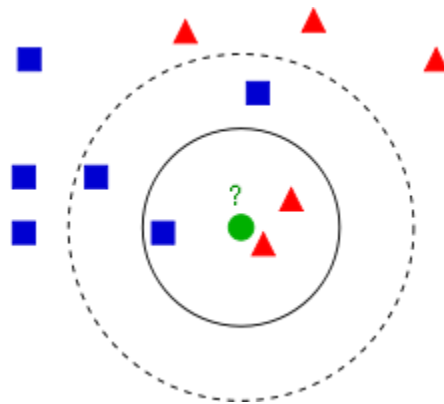
Our image-diff (with L1): **38.6%**

Our image-diff (with L2): **35.4%**

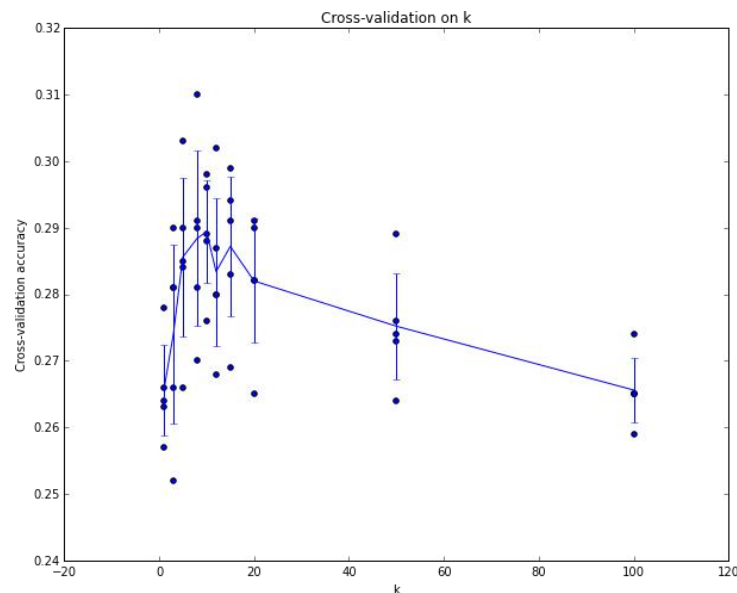
K-Nearest Neighbors: Generalizing the Image-Diff Classifier



Tuning (hyper)parameters:

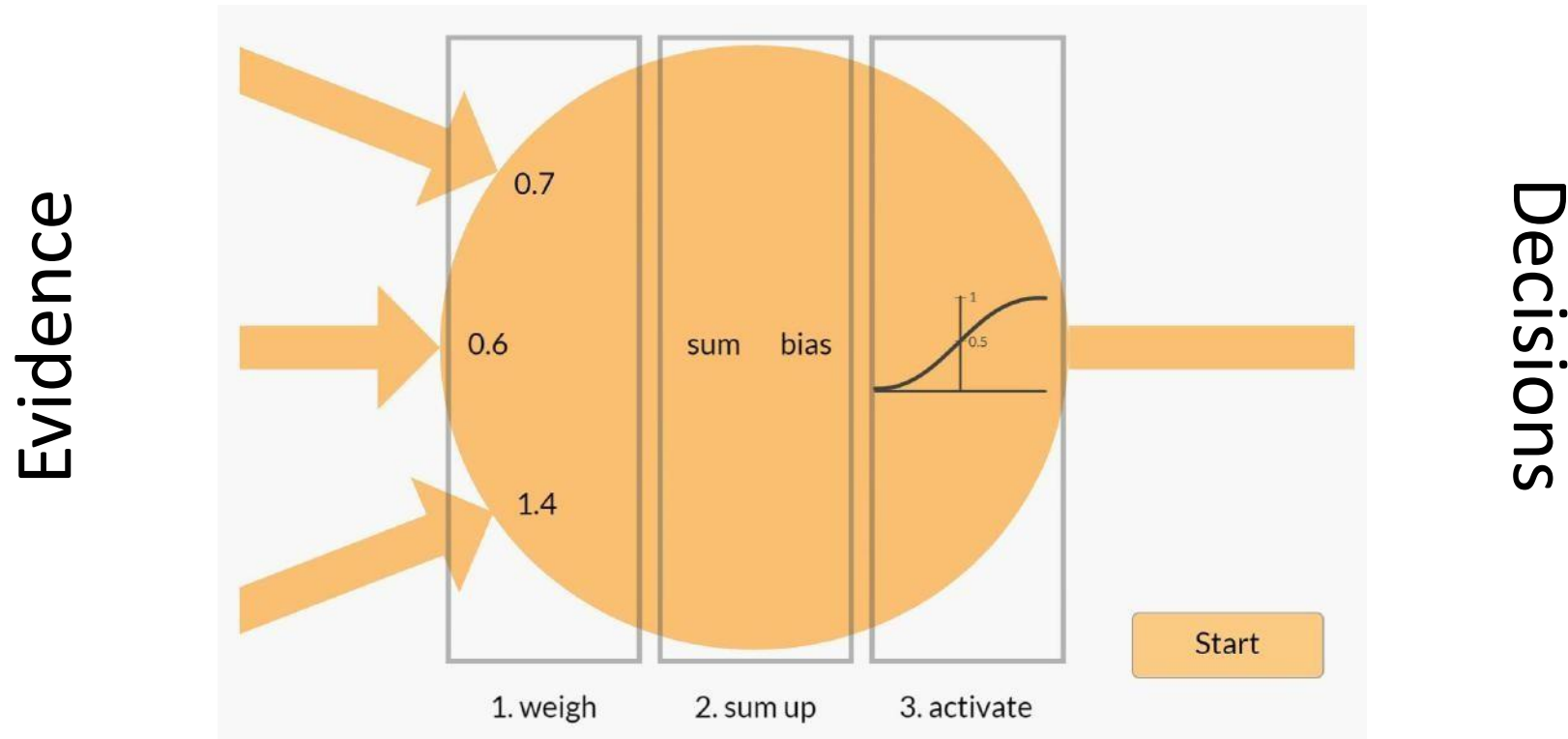


K-Nearest Neighbors: Generalizing the Image-Diff Classifier



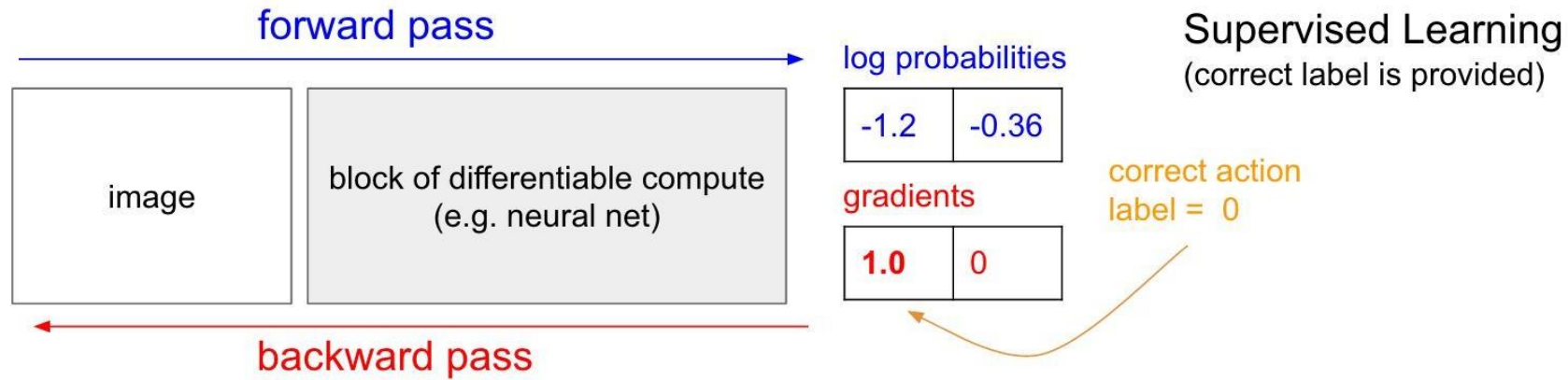
- Accuracy
- Random: 10%
- Training and testing on the same data: 35.4%
- 7-Nearest Neighbors: ~30%
- Human: ~94%
- ...
- Convolutional Neural Networks: ~95%

Reminder: Weighing the Evidence



$$\text{output} = \begin{cases} 0 & \text{if } \sum_j w_j x_j \leq \text{threshold} \\ 1 & \text{if } \sum_j w_j x_j > \text{threshold} \end{cases}$$

Reminder: “Learning” is Optimization of a Function

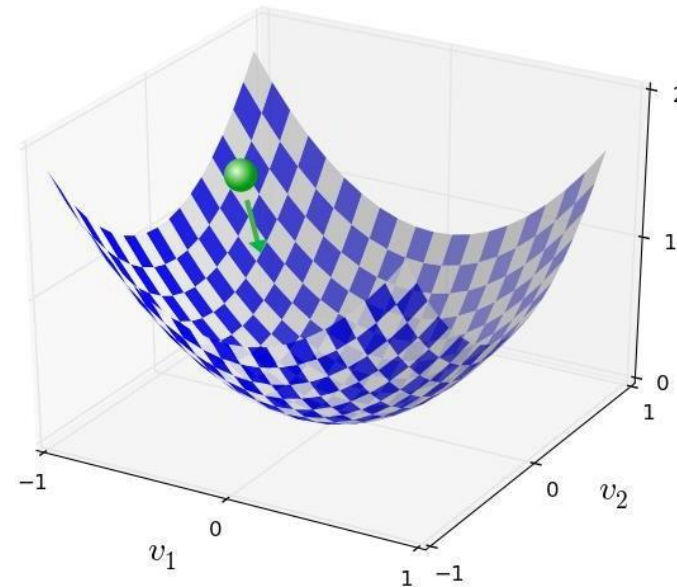


Ground truth for “6”:

$$y(x) = (0, 0, 0, 0, 0, 0, 1, 0, 0, 0)^T$$

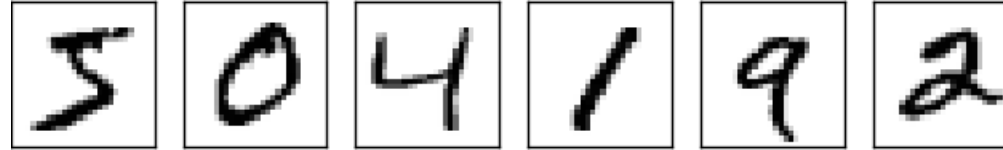
“Loss” function:

$$C(w, b) \equiv \frac{1}{2n} \sum_x \|y(x) - a\|^2$$

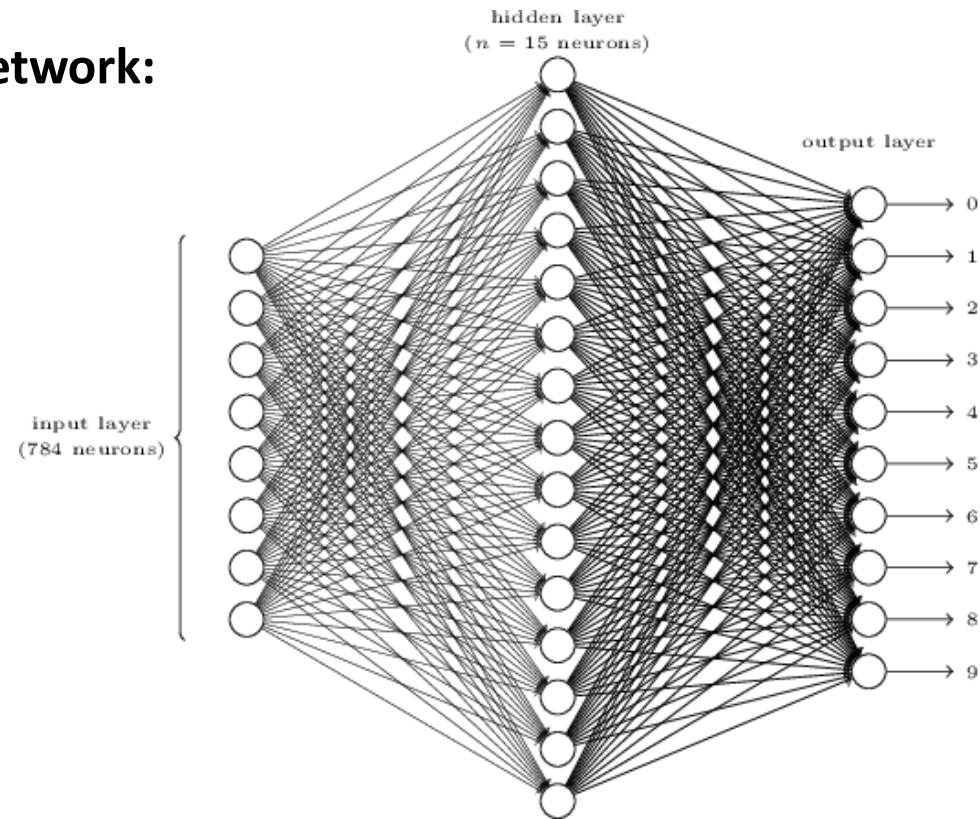


Classify and Image of a Number

Input:
(28x28)

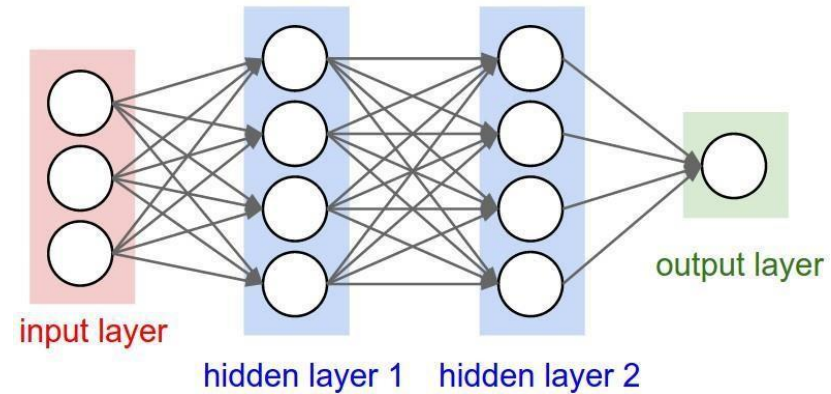


Network:

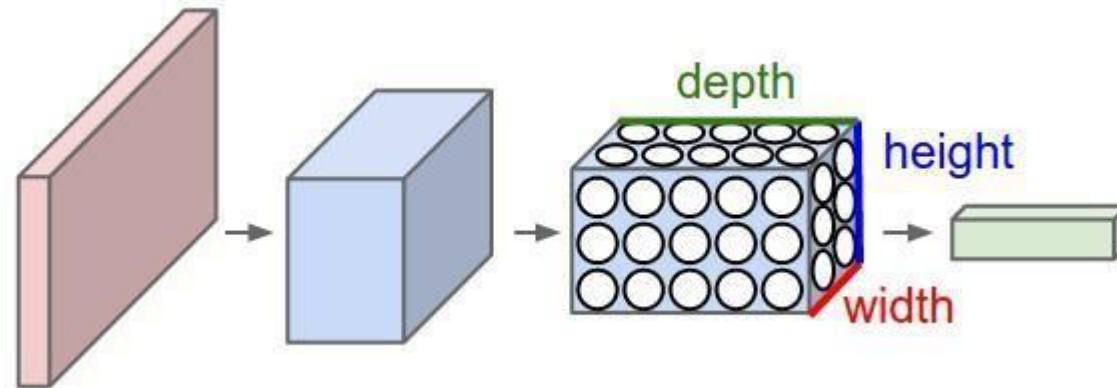


Convolutional Neural Networks

Regular neural network (fully connected):



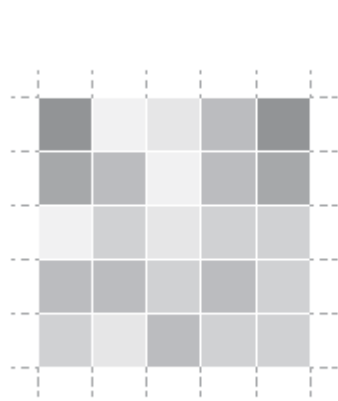
Convolutional neural network:



Each layer takes a 3d volume, produces 3d volume with some smooth function that may or may not have parameters.

Principale of convolution

2D convolution



5	2	1	3	5
4	3	2	3	4
0	2	1	2	2
3	3	2	3	2
2	1	3	2	2

Image piece

5	2	1
4	3	2
0	2	1

x

Kernel 3x3

1	0	1
0	1	0
1	0	1

ω



10

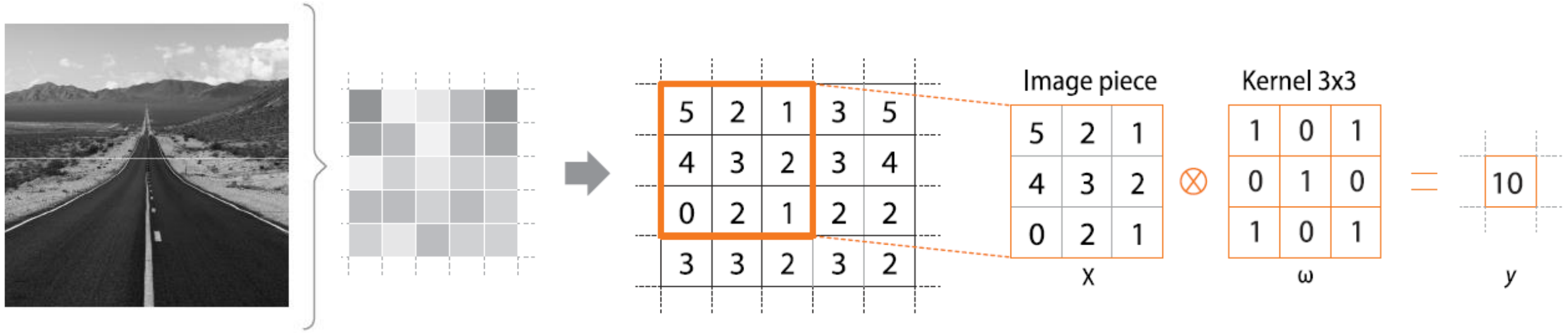
y

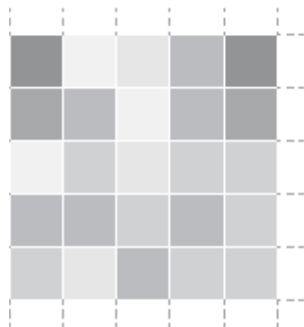
$$\begin{aligned} y &= 5 \times 1 + 2 \times 0 + 1 \times 1 \\ &+ 4 \times 0 + 3 \times 1 + 2 \times 0 \\ &+ 0 \times 1 + 2 \times 0 + 1 \times 1 = 10 \end{aligned}$$

$$y = \sum_{i=1}^n \sum_{j=1}^m x_{i,j} \cdot \omega_{i,j} \quad \text{with} \quad \begin{cases} n & \text{kernel width} \\ m & \text{kernel height} \end{cases}$$

⊗ Hadamard product

Produit de Hadamard : est une opération binaire qui pour deux matrices de mêmes dimensions, associe une autre matrice, de même dimension, et où chaque coefficient est le produit terme à terme des deux matrices.





5	2	1	3	5
4	3	2	3	4
0	2	1	2	2
3	3	2	3	2

Image piece

2	1	3
3	2	3
2	1	2

x

Kernel 3x3

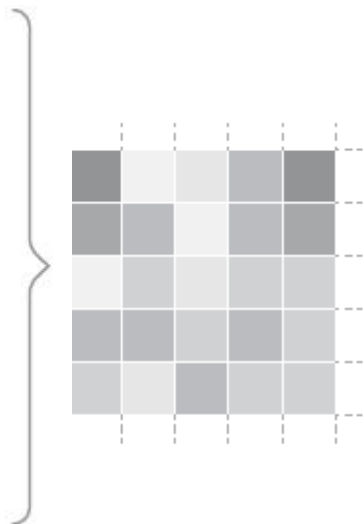
1	0	1
0	1	0
1	0	1

ω



	10	11

y



5	2	1	3	5
4	3	2	3	4
0	2	1	2	2
3	3	2	3	2

Image piece

1	3	5
2	3	4
1	2	2

x



Kernel 3x3

1	0	1
0	1	0
1	0	1

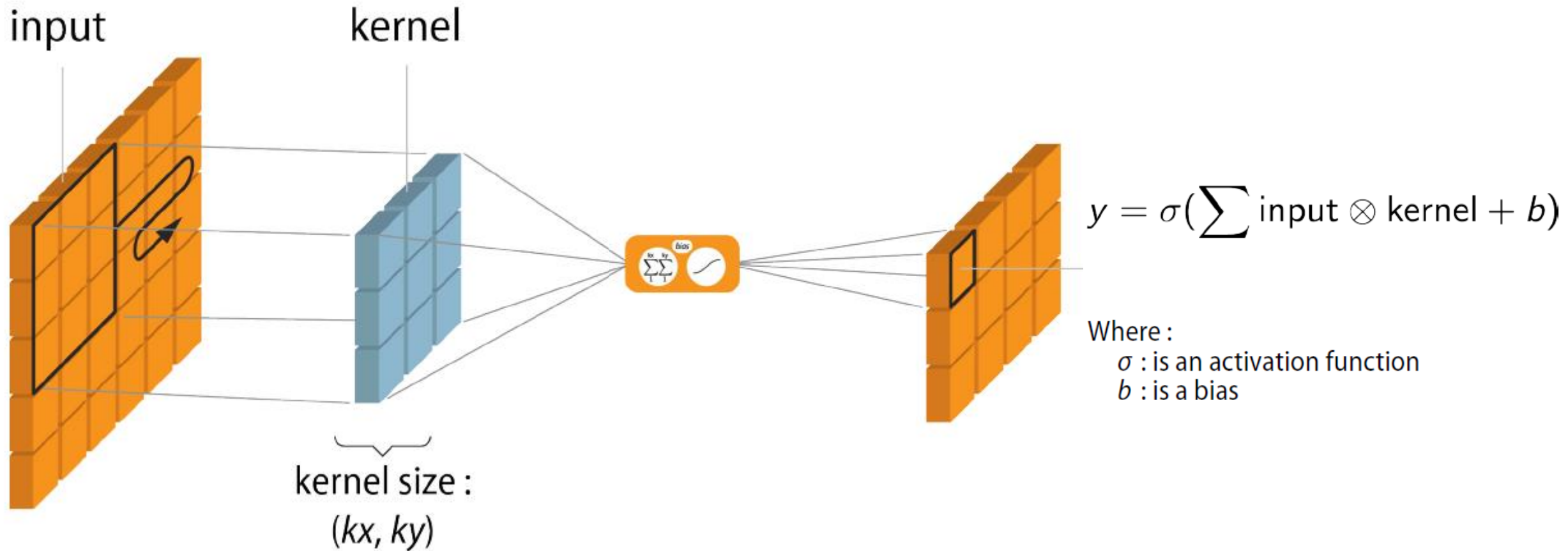
ω



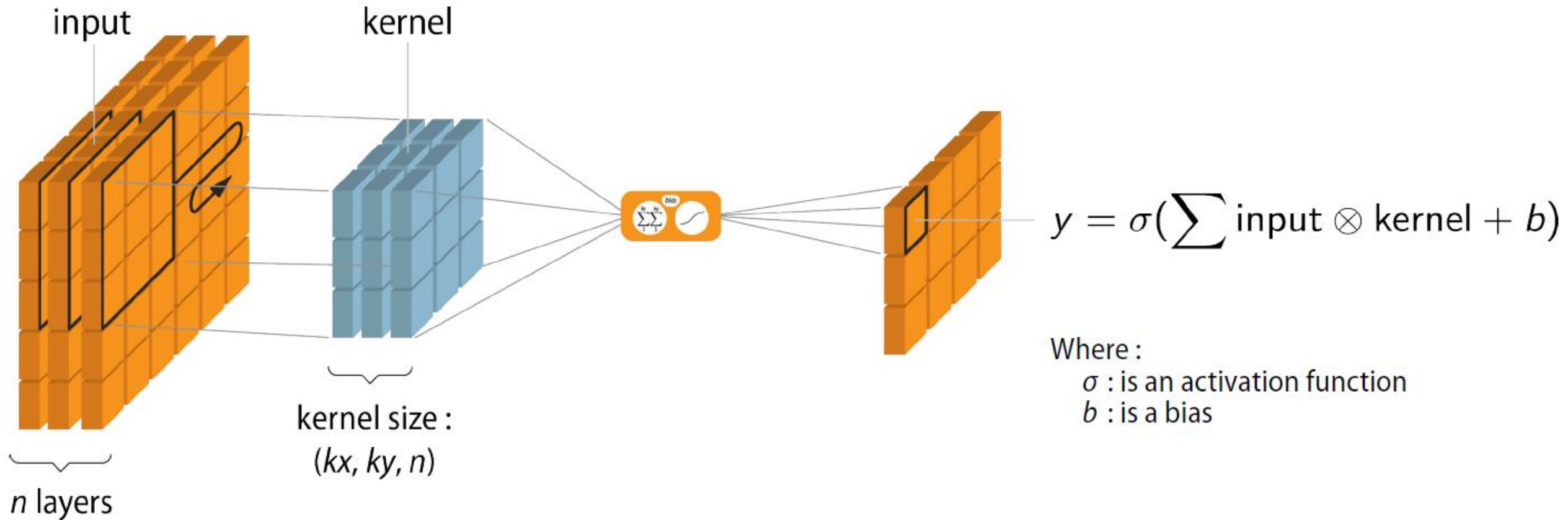
	10	11	12

y

Convolutional layer

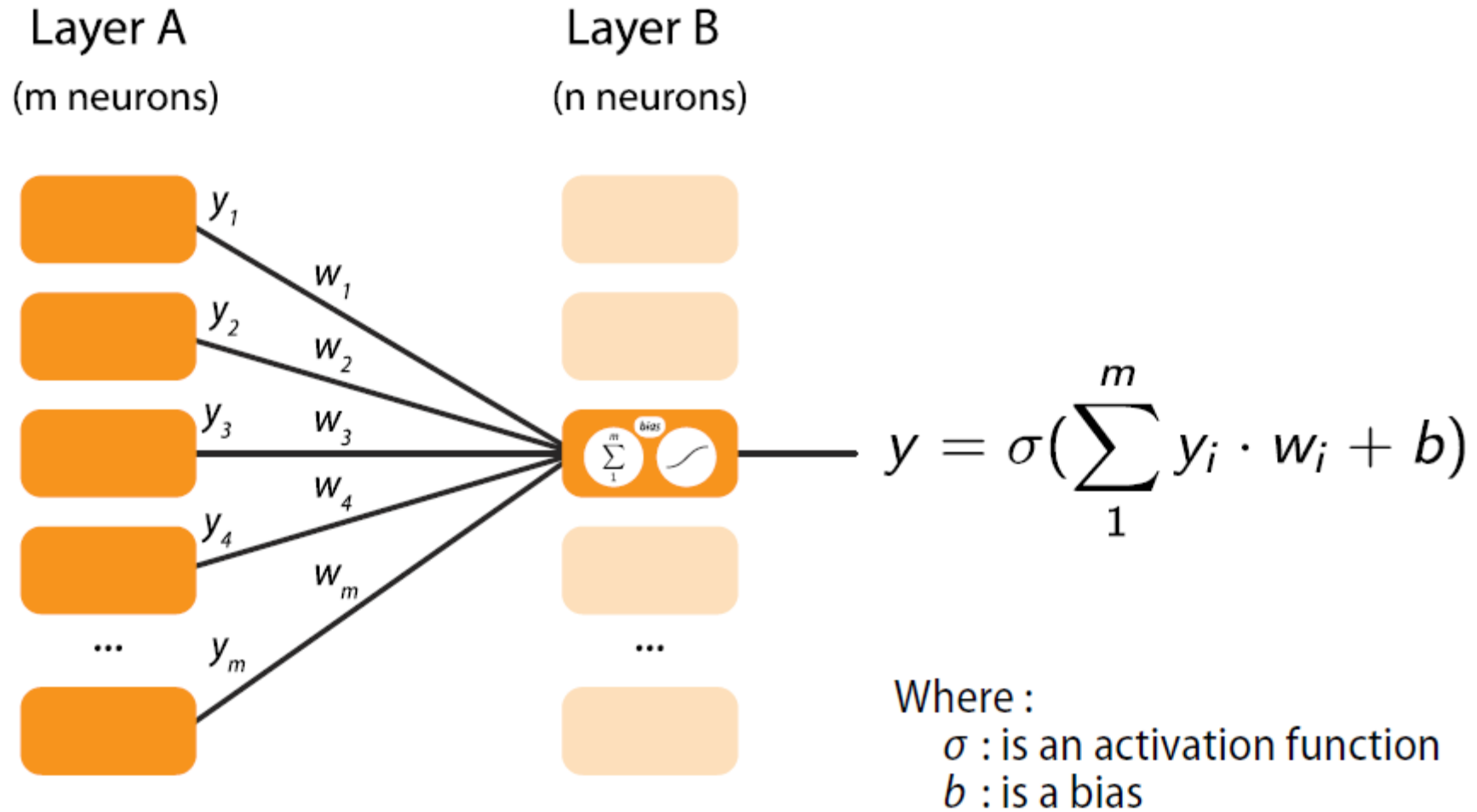


Convolutional layer



m convolutional layers, need m convolutional neurons

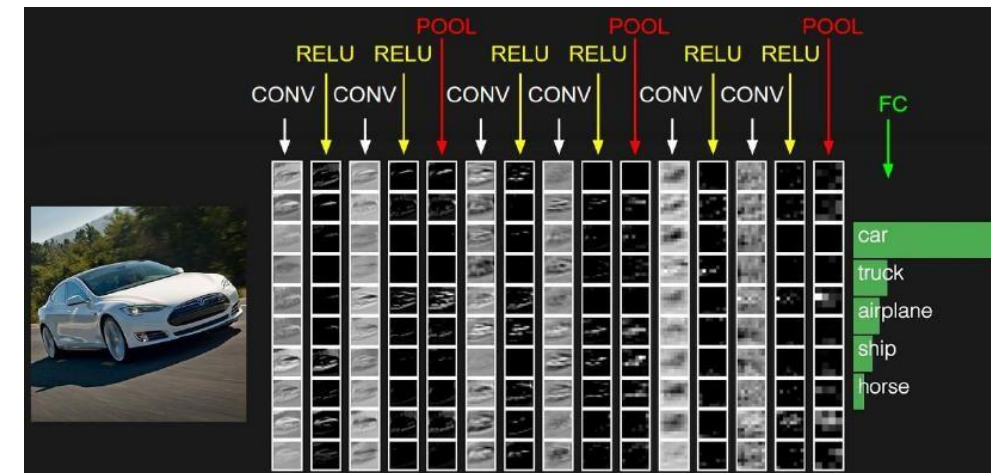
Convolutional layer



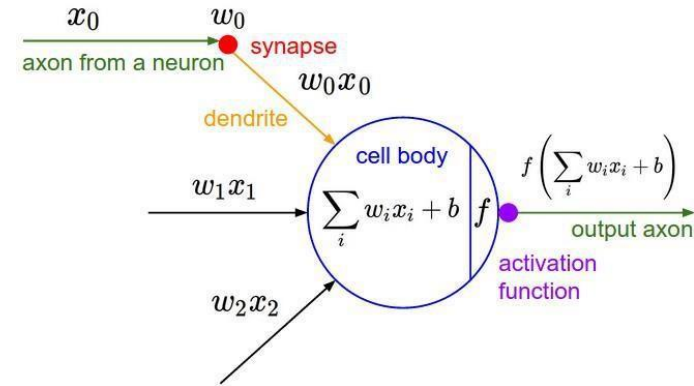
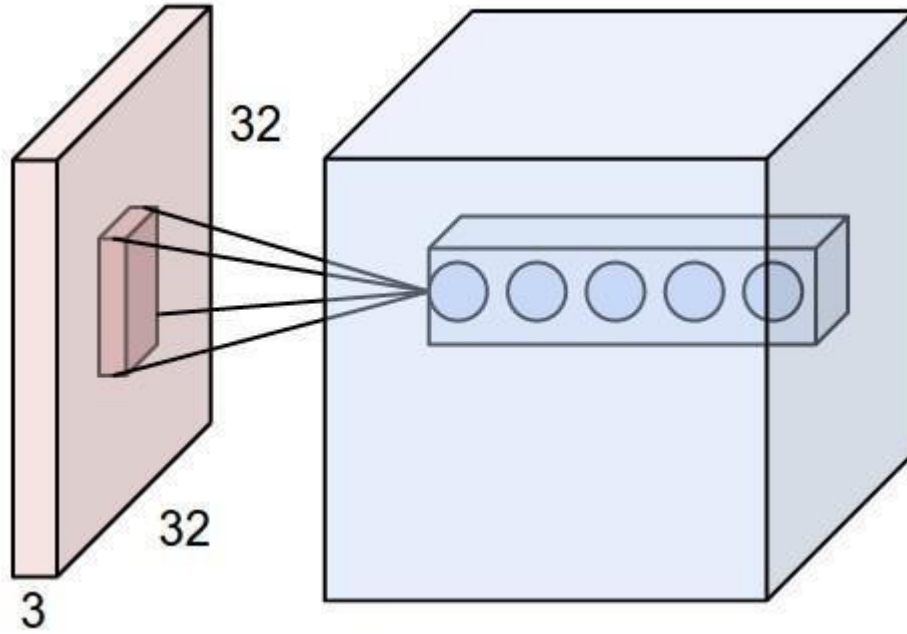
Convolutional Layer

- **INPUT** [32x32x3] will hold the raw pixel values of the image, in this case an image of width 32, height 32, and with three color channels R,G,B.
- **CONV** layer will compute the output of neurons that are connected to local regions in the input, each computing a dot product between their weights and a small region they are connected to in the input volume. This may result in volume such as [32x32x12] if we decided to use 12 filters.
- **RELU** layer will apply an elementwise activation function, such as the $\max(0, x)$ thresholding at zero. This leaves the size of the volume unchanged ([32x32x12]).
- **POOL** layer will perform a downsampling operation along the spatial dimensions (width, height), resulting in volume such as [16x16x12].
- **FC** (i.e. fully-connected) layer will compute the class scores, resulting in volume of size [1x1x10], where each of the 10 numbers correspond to a class score, such as among the 10 categories of CIFAR-10. As with ordinary Neural Networks and as the name implies, each neuron in this layer will be connected to all the numbers in the previous volume.

Layers **highlighted in blue**
have learnable parameters.



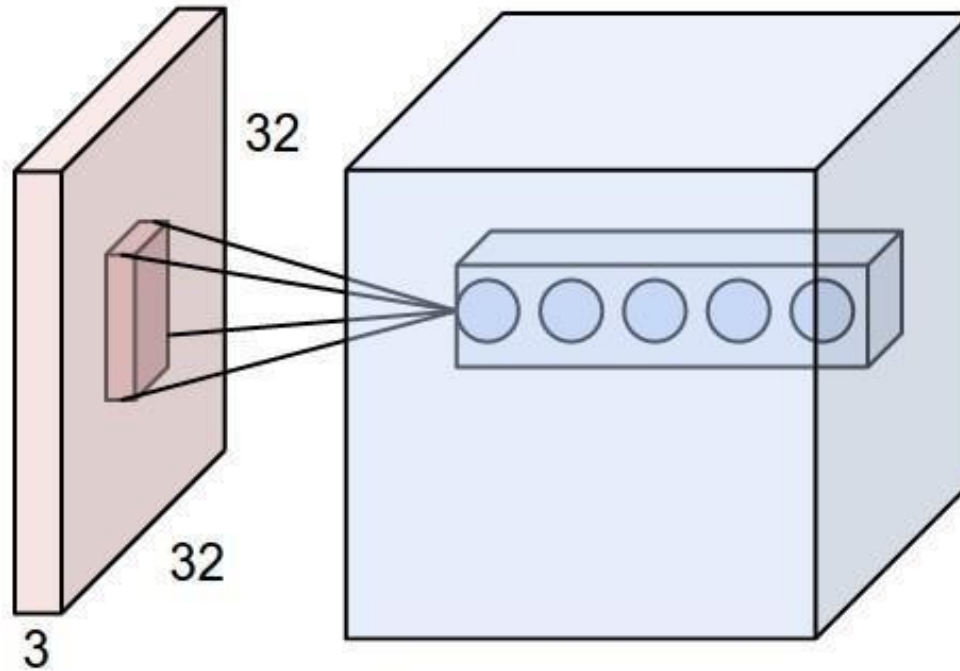
Dealing with Images: Local Connectivity



Same neuron. Just more focused (narrow “receptive field”).

The parameters on a each filter are spatially “shared”
(if a feature is useful in one place, it’s useful elsewhere)

ConvNets: Spatial Arrangement of Output Volume

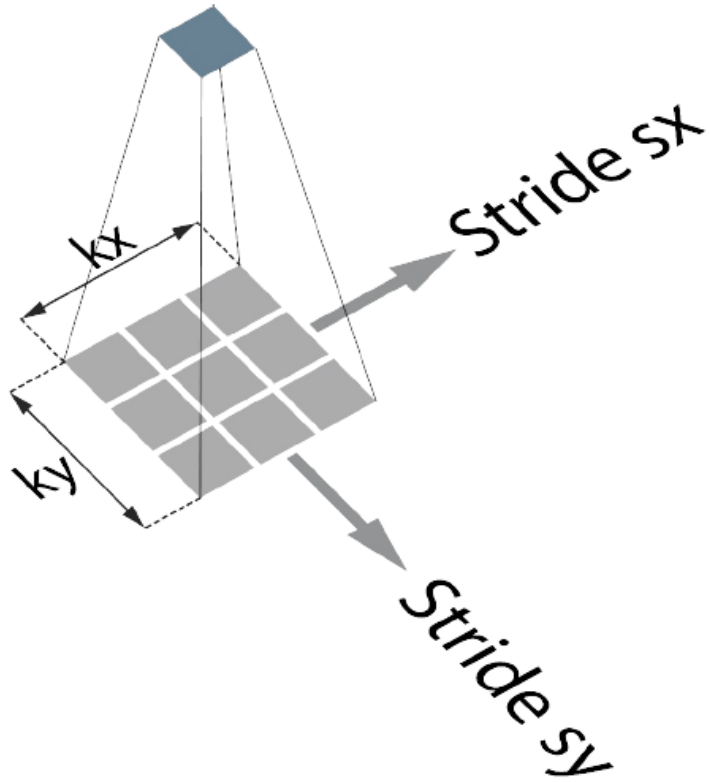


- **Depth:** number of filters
- **Stride:** filter step size (when we “slide” it)
- **Padding:** zero-pad the input

Parameters of the CNN

Strides(pas) & Padding (bourrage)

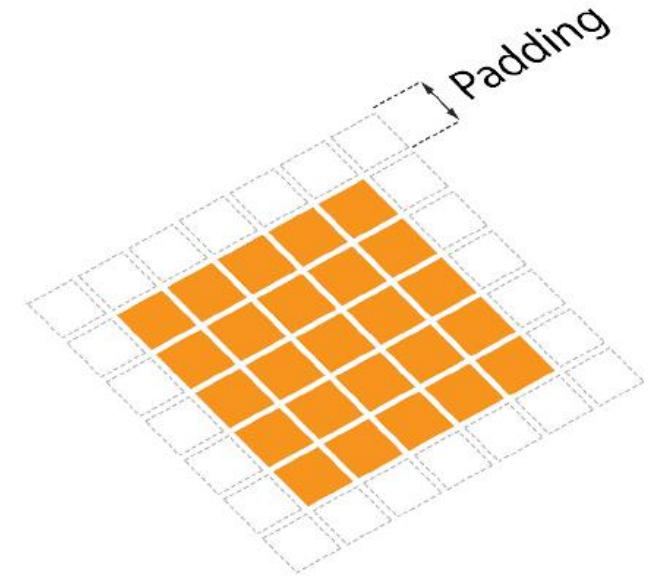
- ❑ Strides : denotes how many steps we are moving in each steps in convolution



kernel size

strides

- ❑ Padding : is a process of adding zeros to the input matrix symmetrically



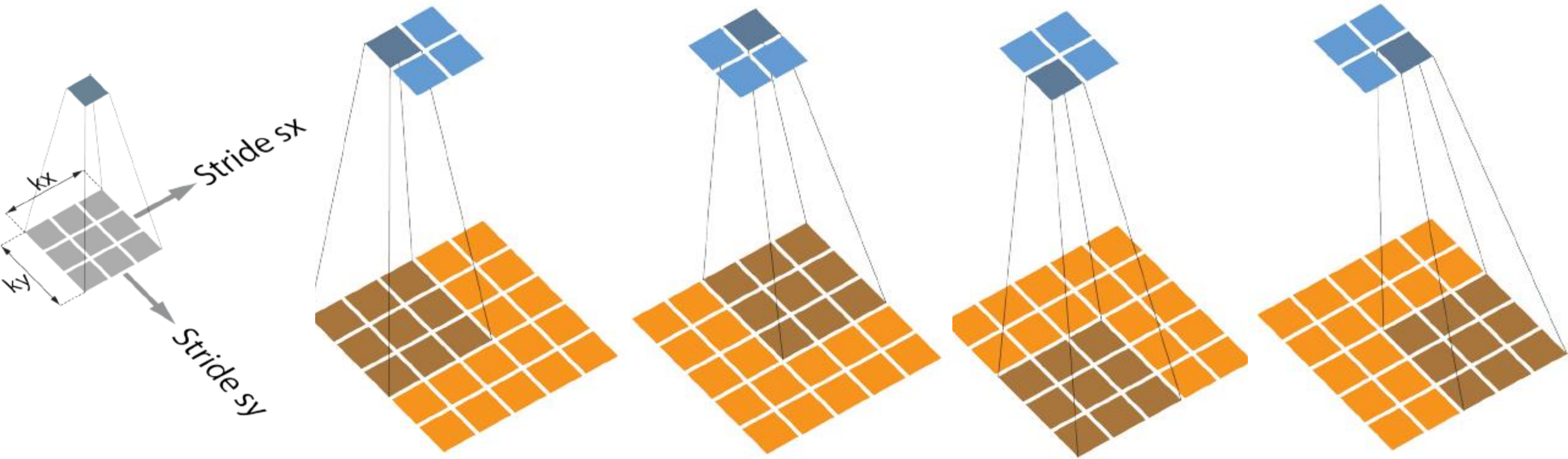
padding

'same'

'valid'

Parameters of the CNN

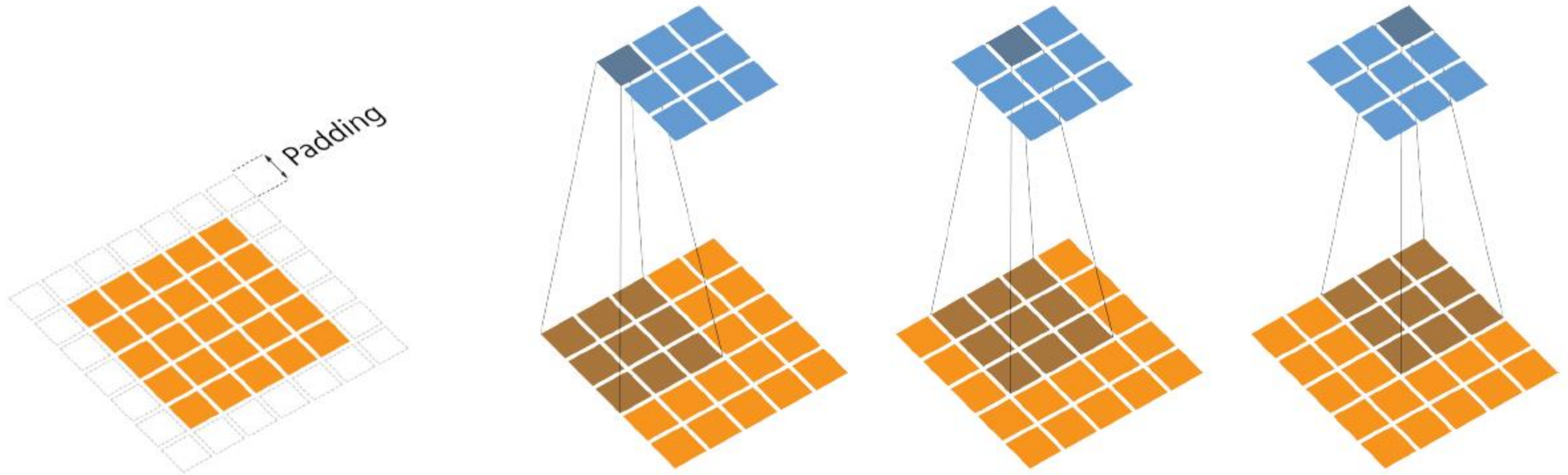
Strides



$\text{strides} = (s_x, s_y) \Rightarrow$ A $\text{strides}=(2,2)$ will reduce by 2 the output size

Parameters of the CNN

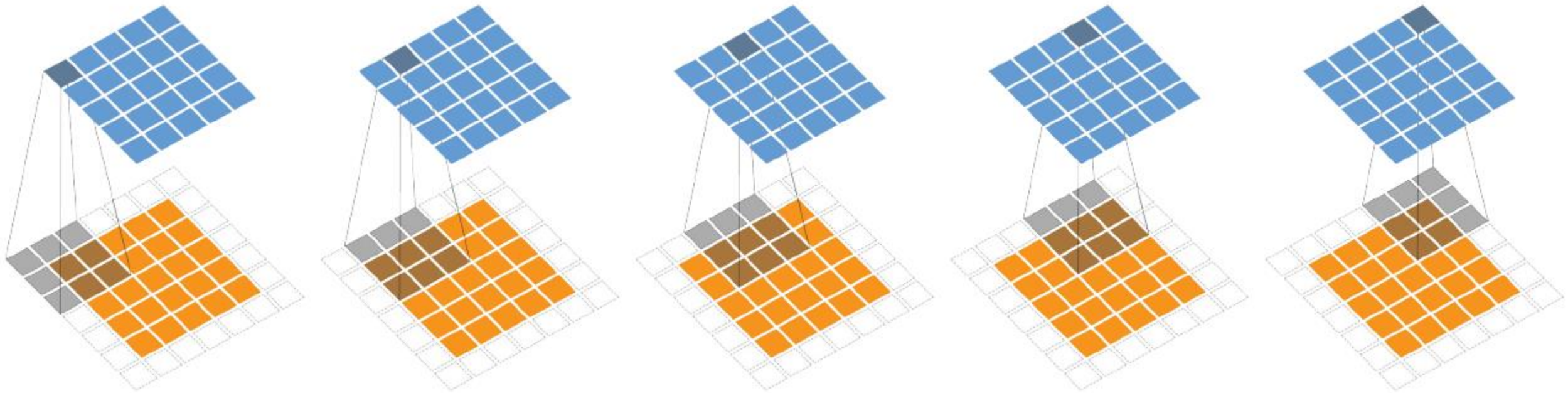
Padding



Keras : padding = 'valid' $\Rightarrow$ Size is not preserved (no padding)

Parameters of the CNN

Padding



Keras : padding = 'same' $\Rightarrow$ size is preserved

Input Volume (+pad 1) (7x7x3)

 $x[:, :, 0]$

0	0	0	0	0	0	0
0	1	2	0	0	0	0
0	2	2	0	0	0	0

0	2	2	2	1	0	0
0	1	0	0	2	0	0
0	1	1	0	1	0	0
0	0	0	0	0	0	0

 $x[:, :, 1]$

0	0	0	0	0	0	0
0	1	0	0	1	0	0
0	1	0	0	0	1	0

0	0	0	1	2	0	0
0	0	1	0	1	0	0
0	1	1	1	1	2	0
0	0	0	0	0	0	0

 $x[:, :, 2]$

0	0	0	0	0	0	0
0	0	0	2	2	2	0
0	1	2	2	2	0	0

0	1	1	1	1	0	0
0	1	2	0	0	0	0
0	2	2	2	1	0	0
0	0	0	0	0	0	0

Filter W0 (3x3x3)

 $w0[:, :, 0]$

0	0	-1
-1	0	0
-1	-1	-1

 $w0[:, :, 1]$

1	1	-1
0	0	0
-1	1	1

 $w0[:, :, 2]$

-1	-1	-1
0	1	1
0	-1	0

Bias b0 (1x1x1)

 $b0[:, :, 0]$

1

Filter W1 (3x3x3)

 $w1[:, :, 0]$

-1	0	0
1	-1	-1
0	0	-1

 $w1[:, :, 1]$

-1	0	1
1	-1	1
-1	0	1

 $w1[:, :, 2]$

-1	-1	-1
-1	1	-1
0	1	-1

Bias b1 (1x1x1)

 $b1[:, :, 0]$

0

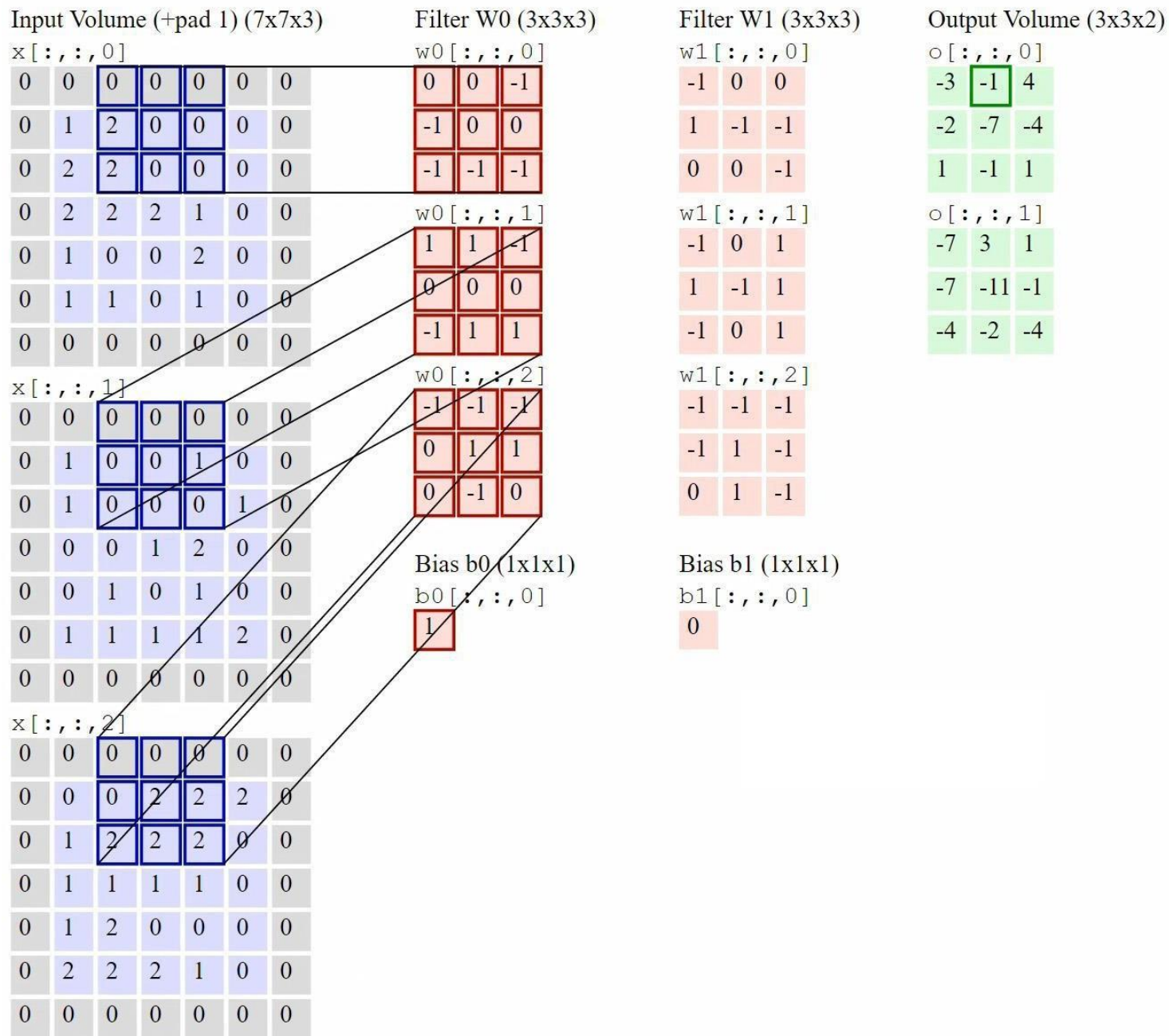
Output Volume (3x3x2)

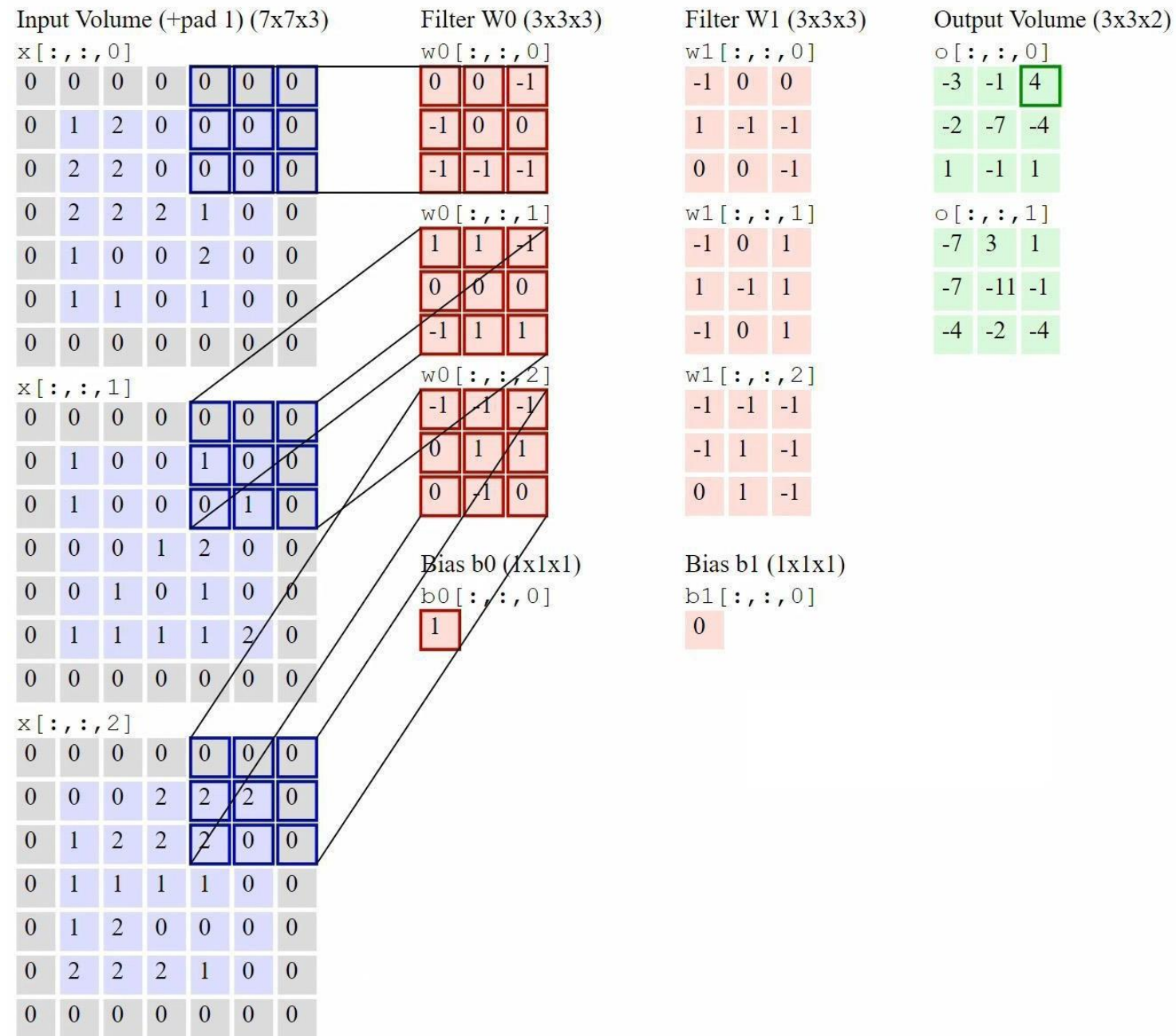
 $o[:, :, 0]$

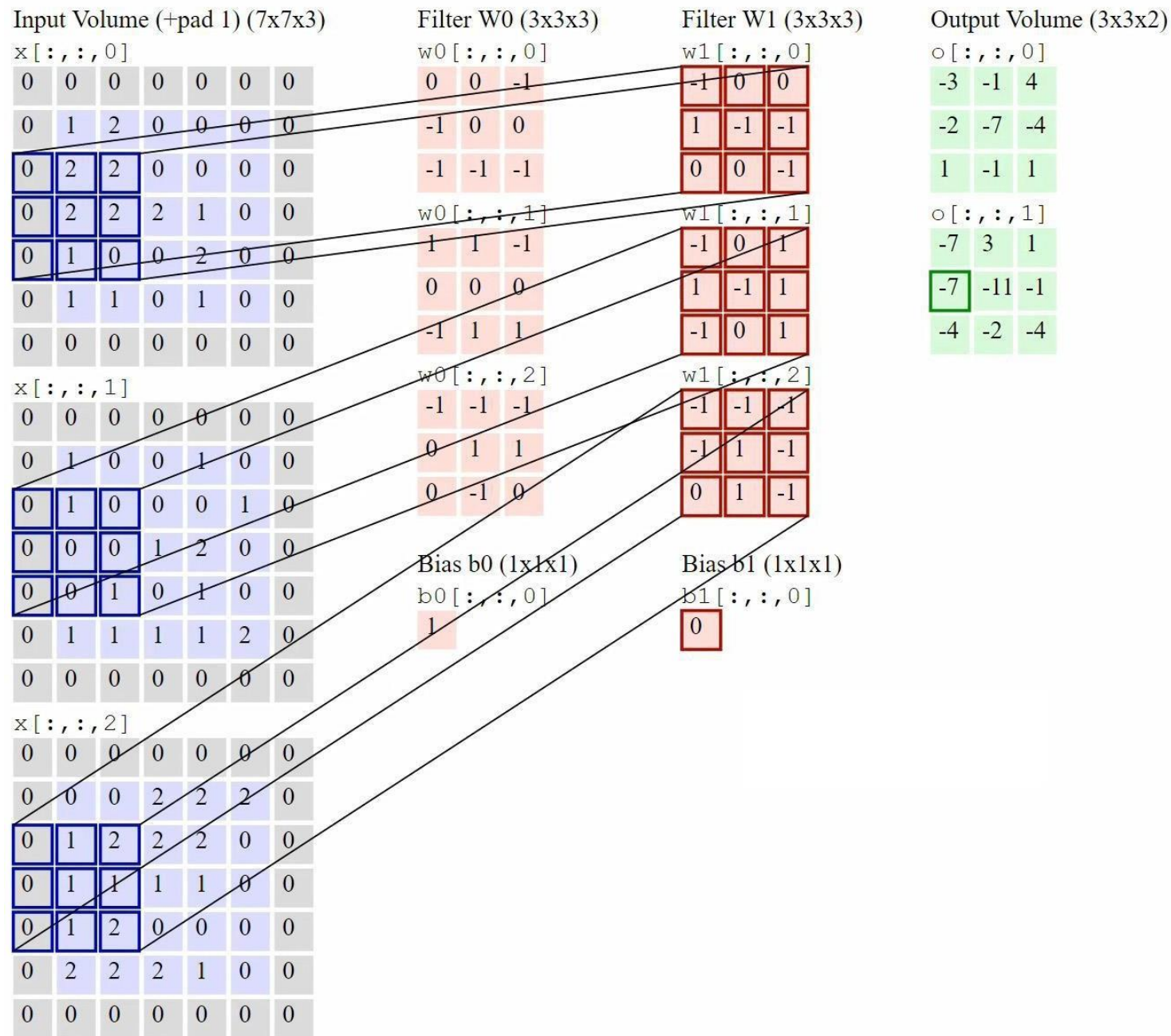
-3	-1	4
-2	-7	-4
1	-1	1

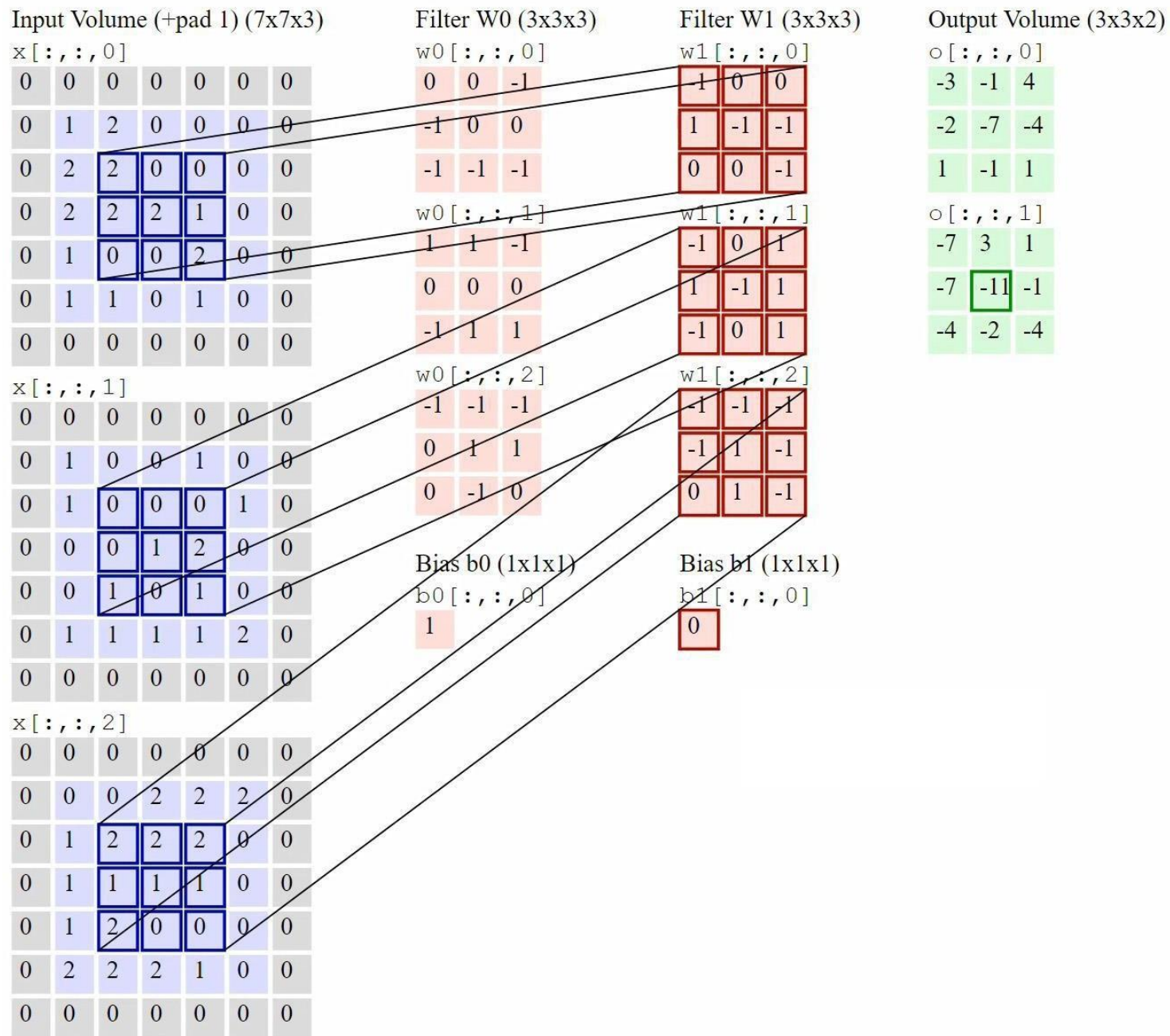
 $o[:, :, 1]$

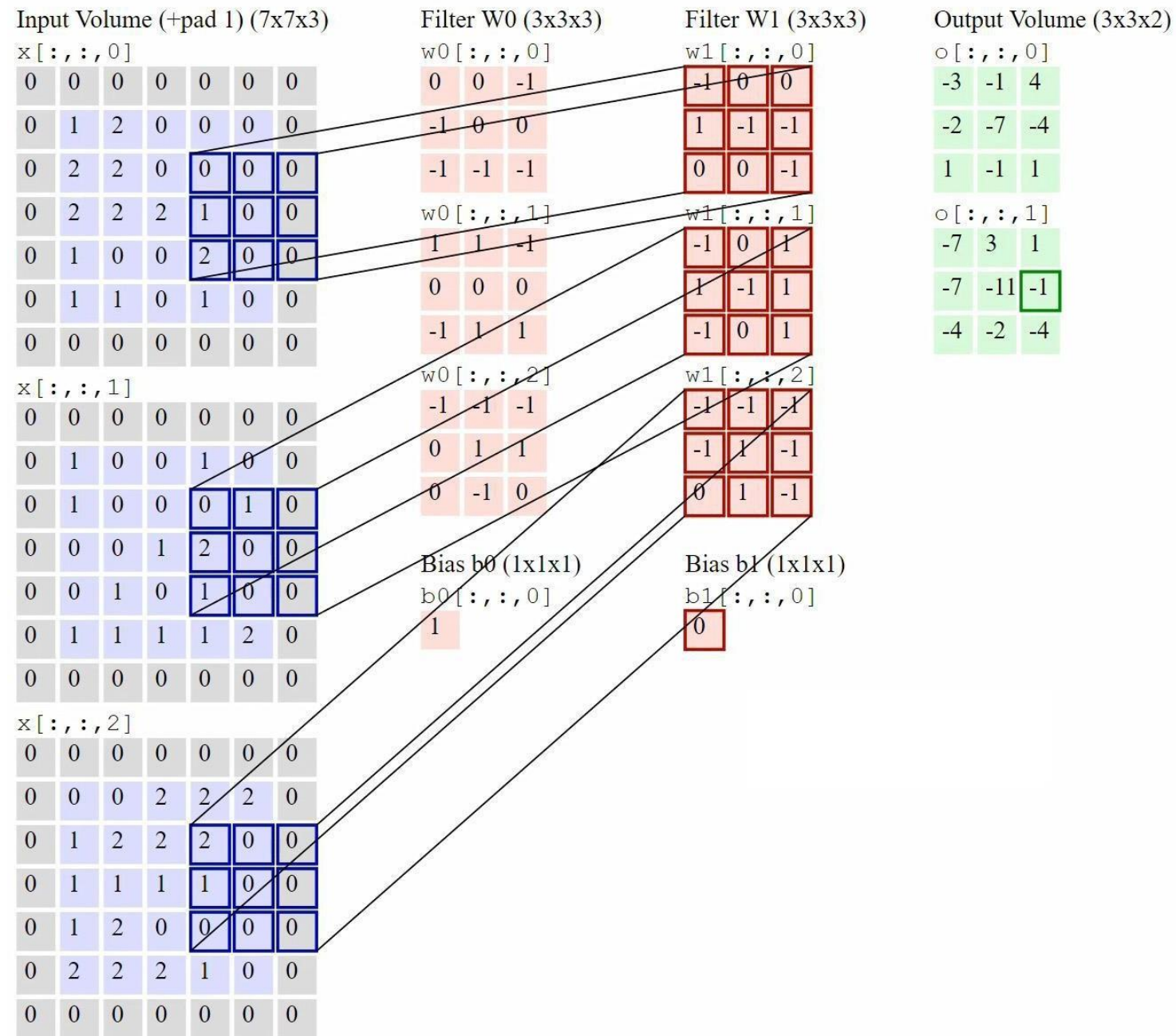
-7	3	1
-7	-11	-1
-4	-2	-4

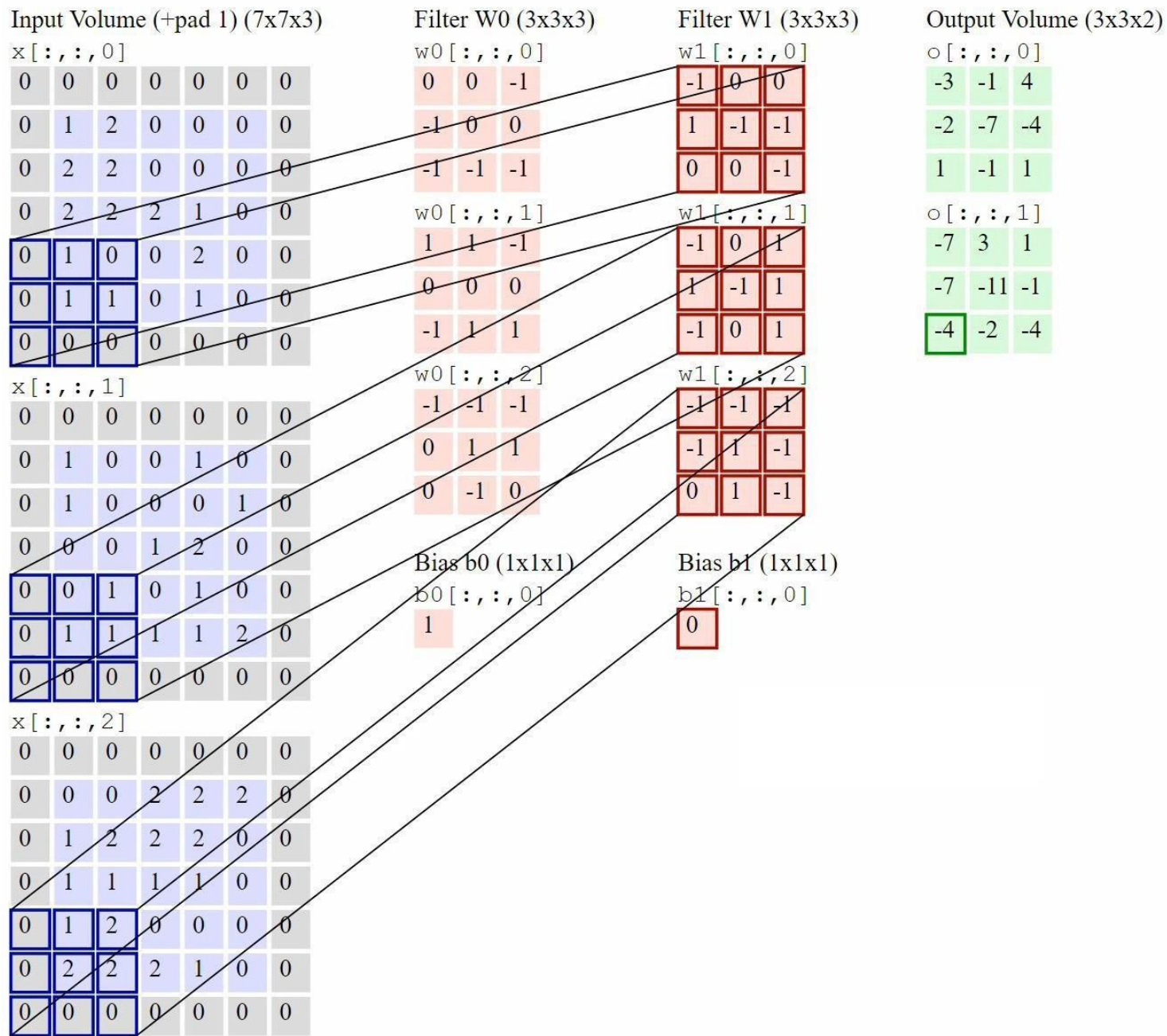


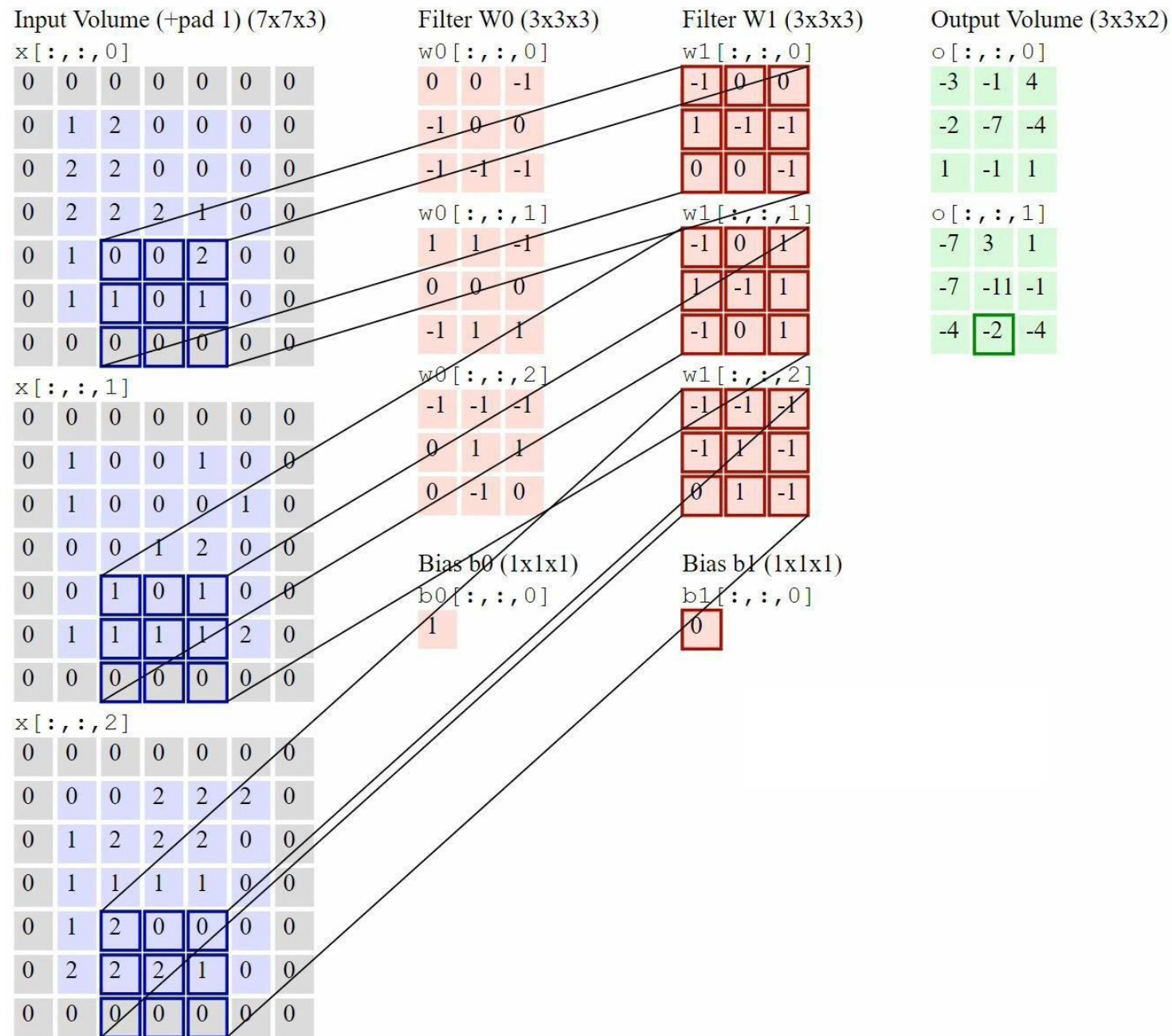


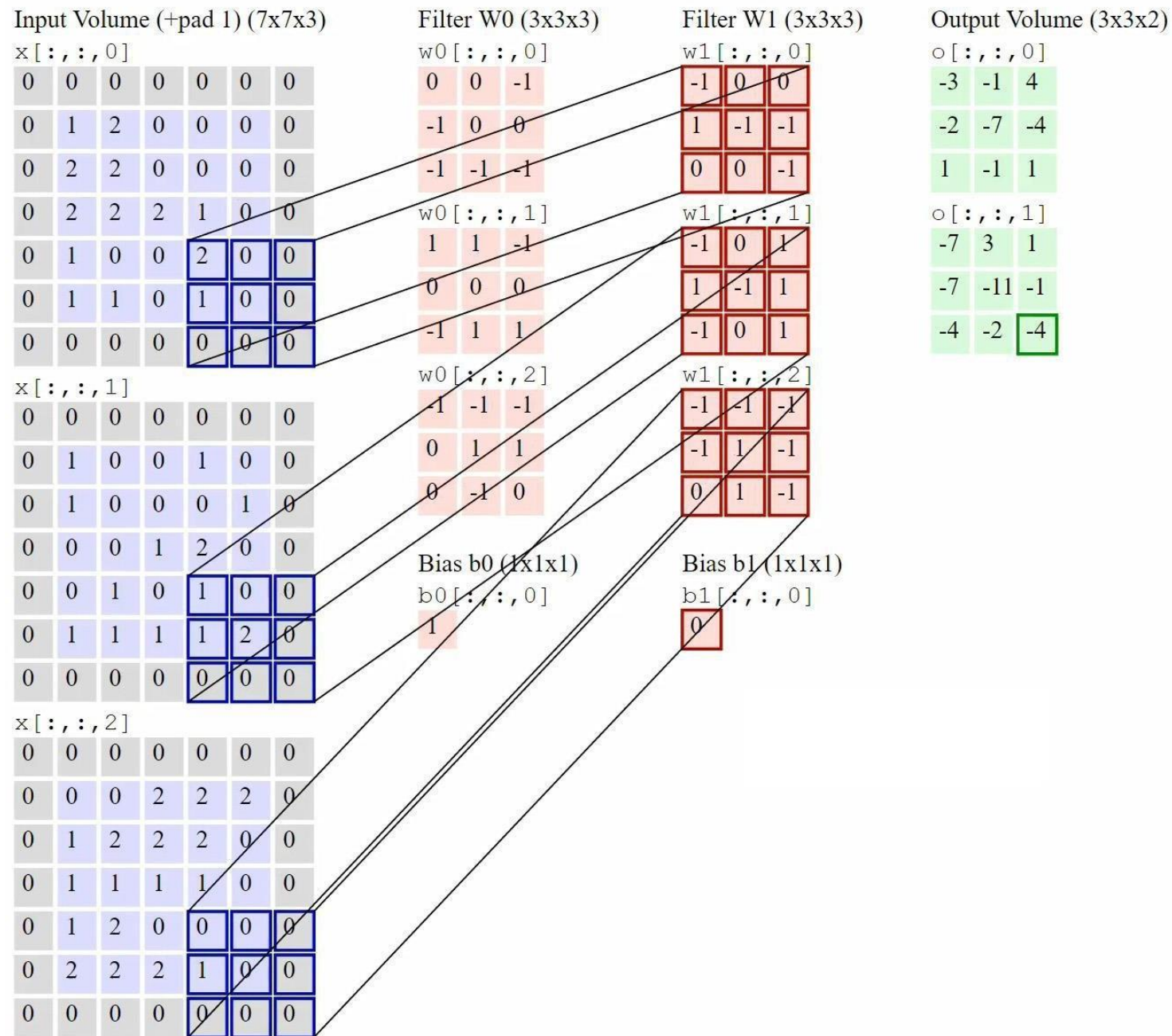




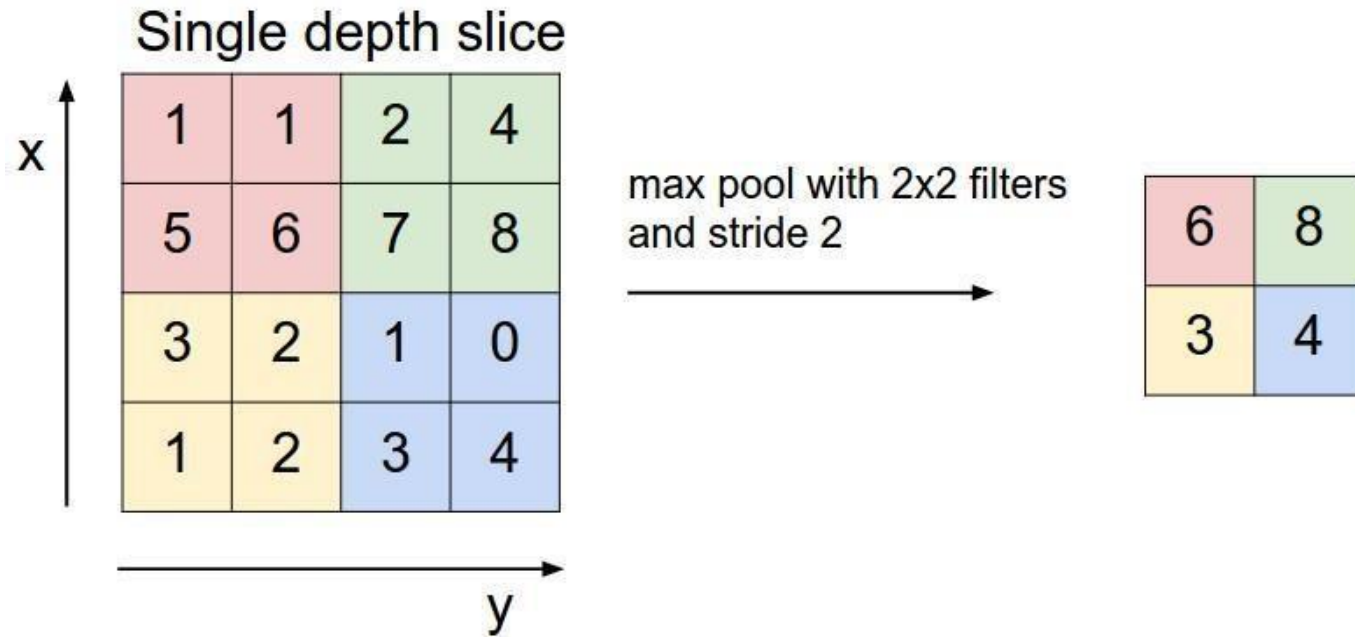




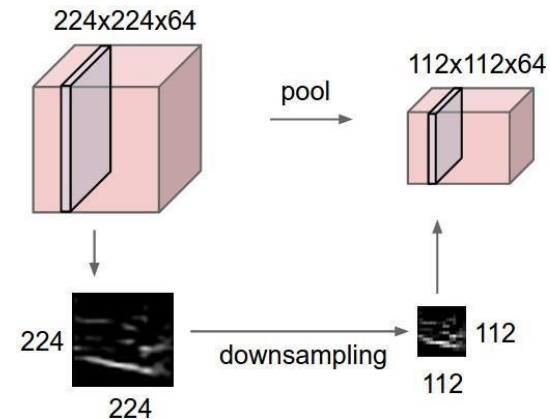




ConvNets: Pooling

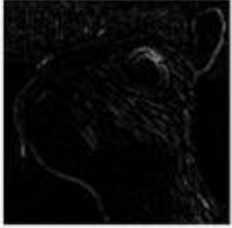


- ❑ Le max-pooling sélectionne dans chaque région de l'image l'information la plus pertinente

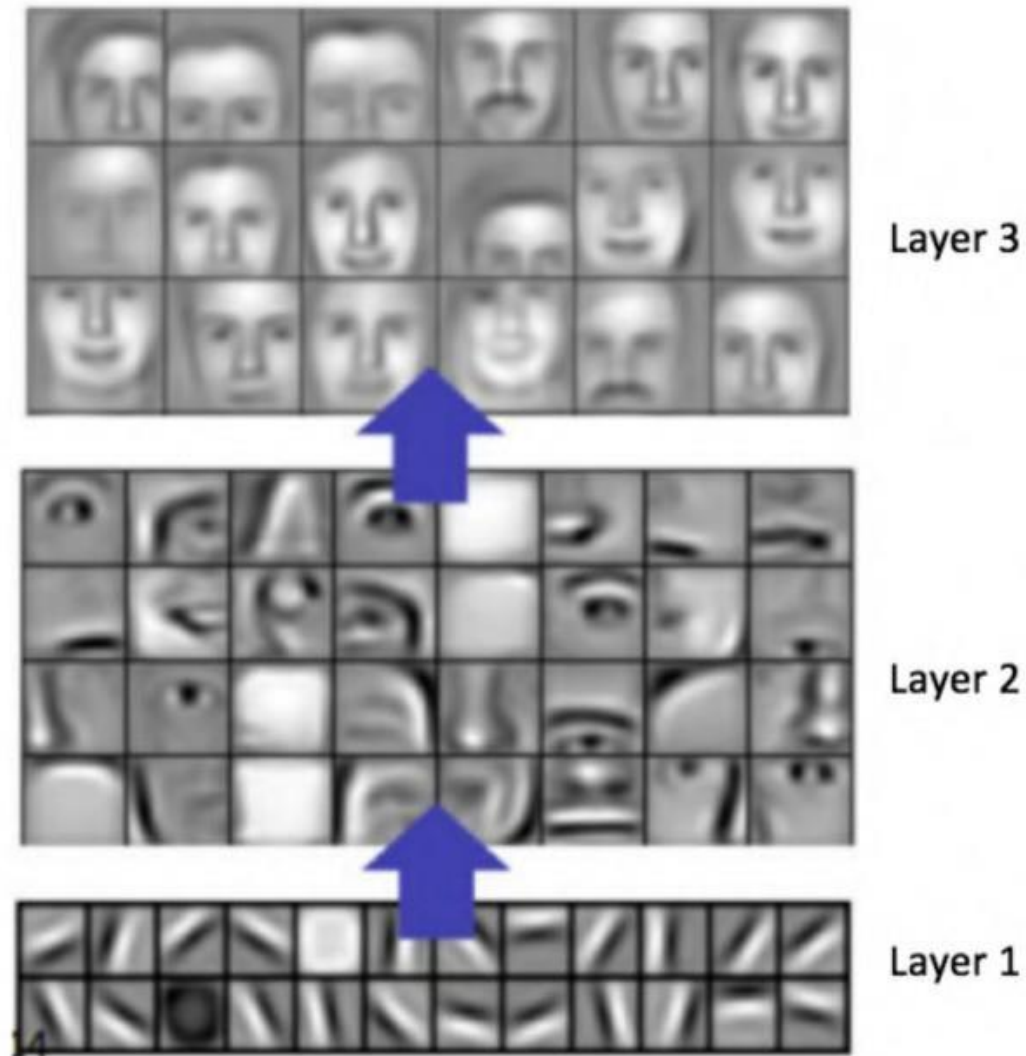


Convolution

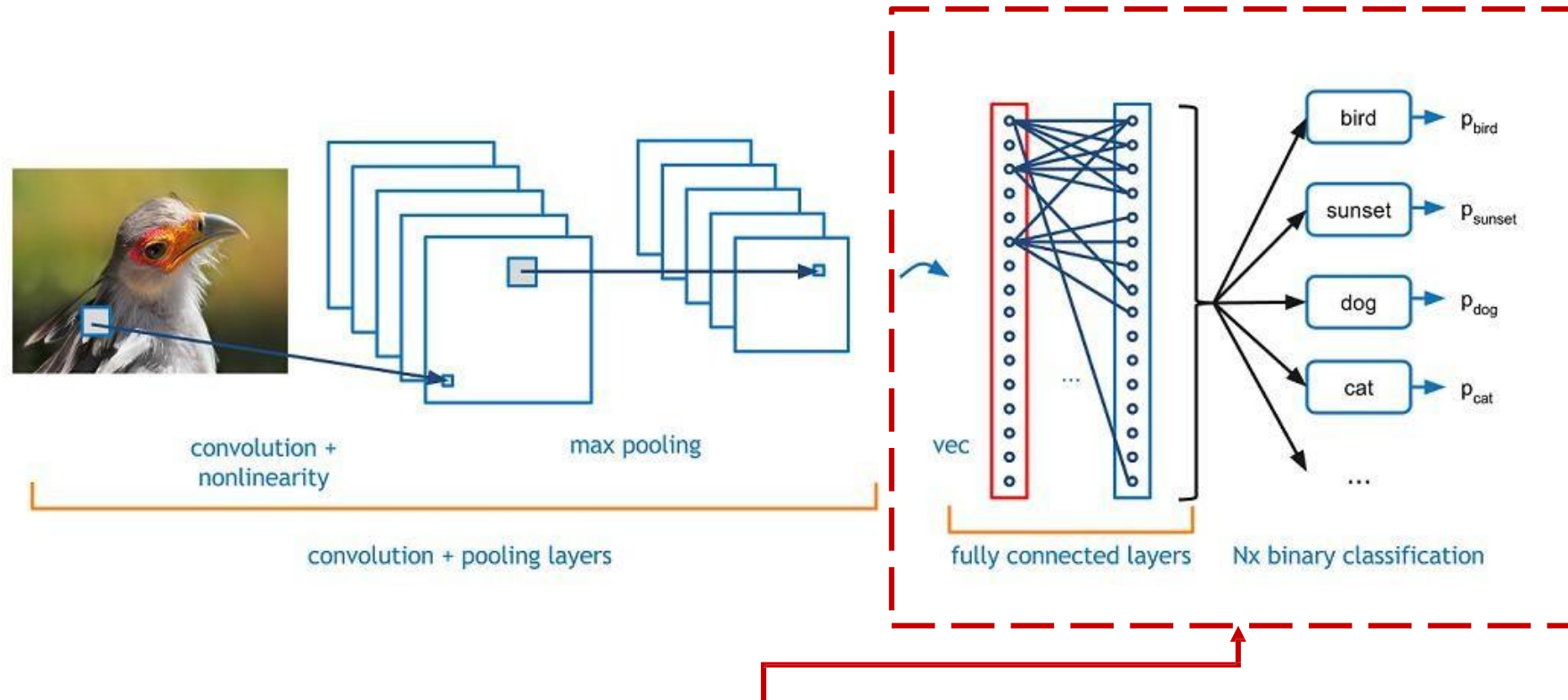


Operation	Filter	Convolved Image
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	

Convolution: Representation Learning



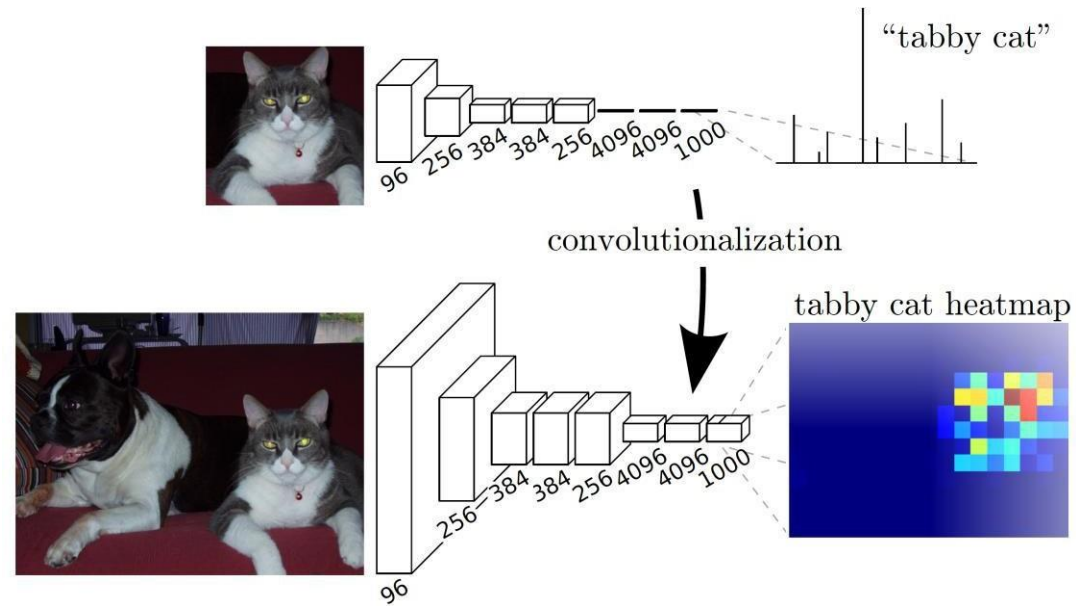
Same Architecture, Many Applications



This part might look different for:

- Different image classification **domains**
- Image captioning with **recurrent neural networks**
- Image object localization with **bounding box**
- Image segmentation with **fully convolutional networks**
- Image segmentation with **deconvolution layers**

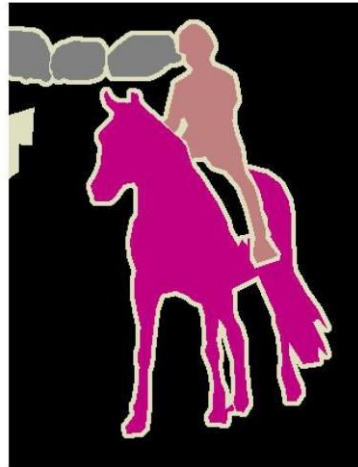
Segmentation



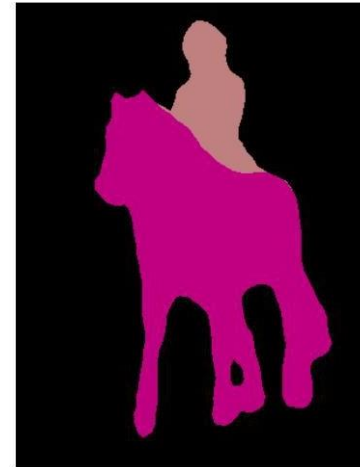
Original



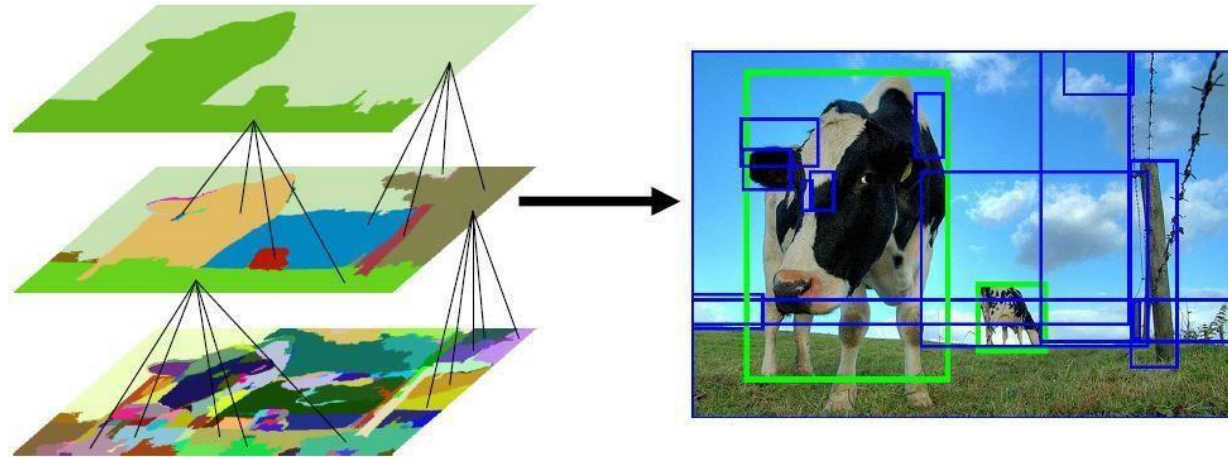
Ground Truth



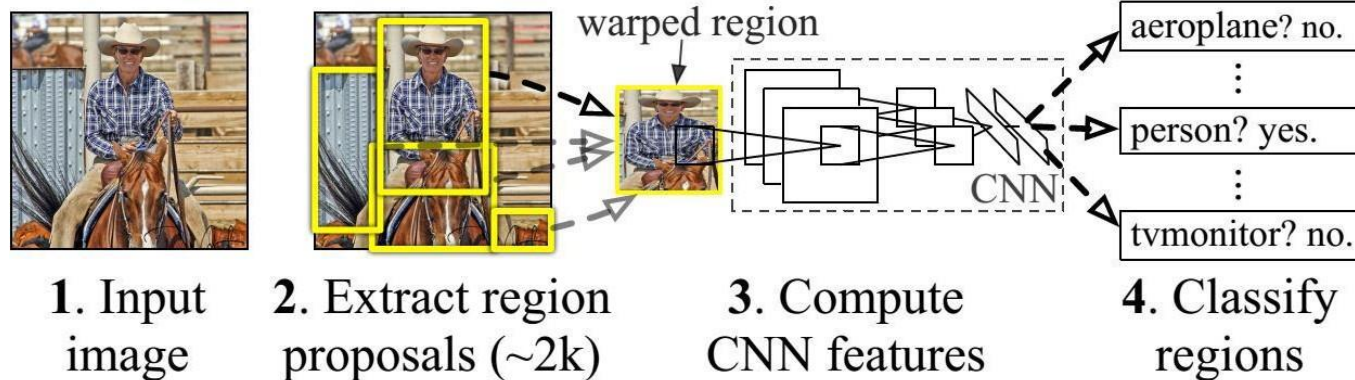
FCN-8



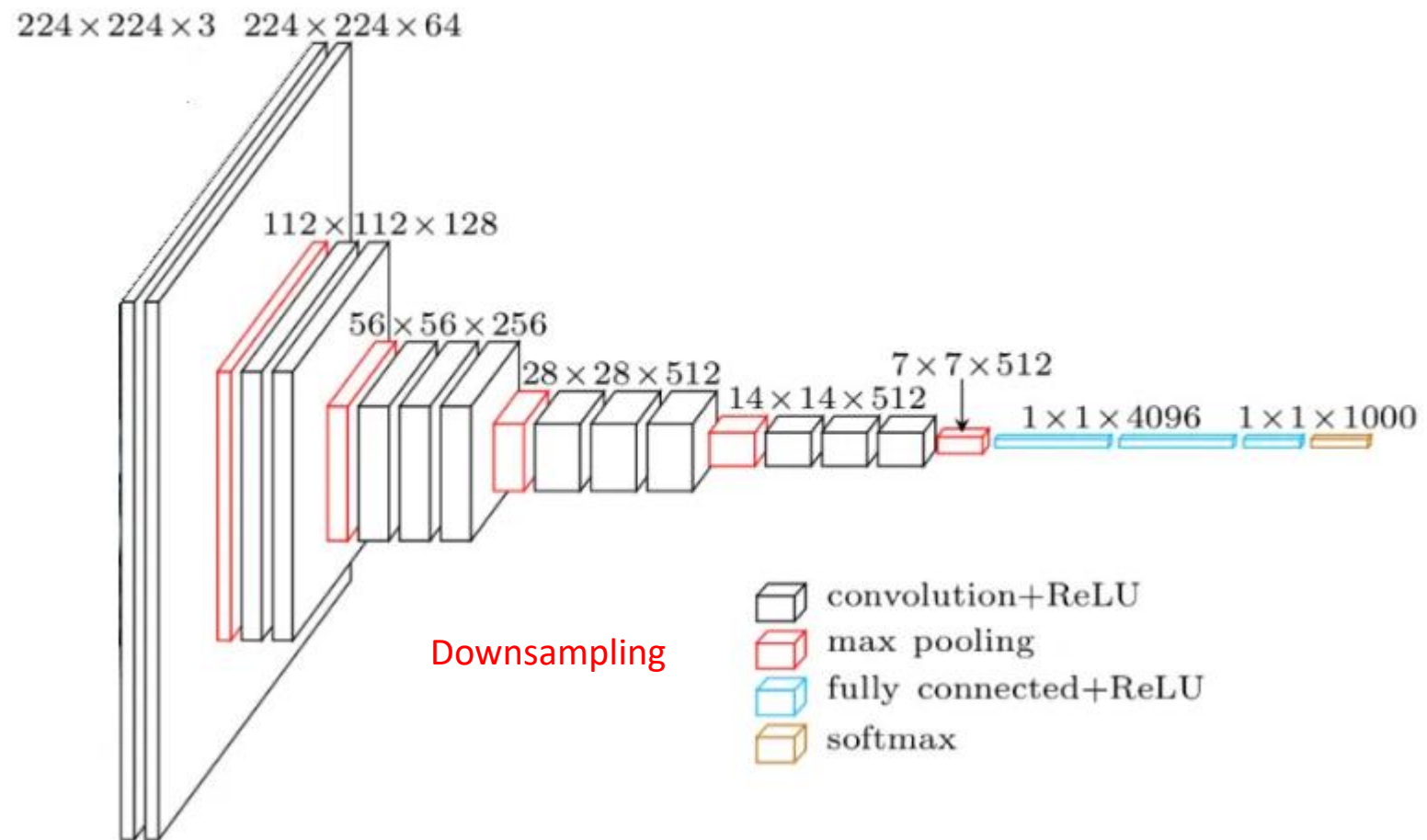
Object Detection



R-CNN: *Regions with CNN features*



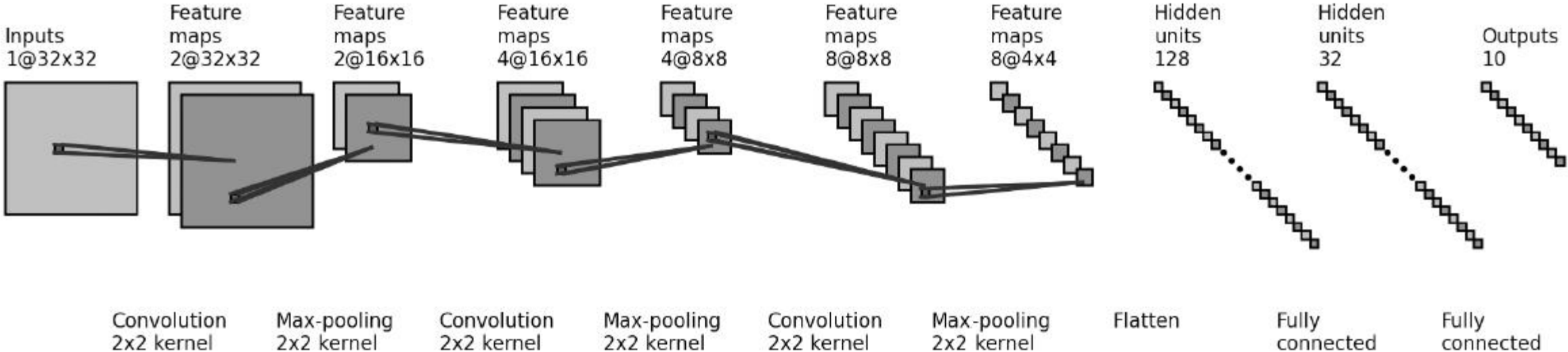
Practical case

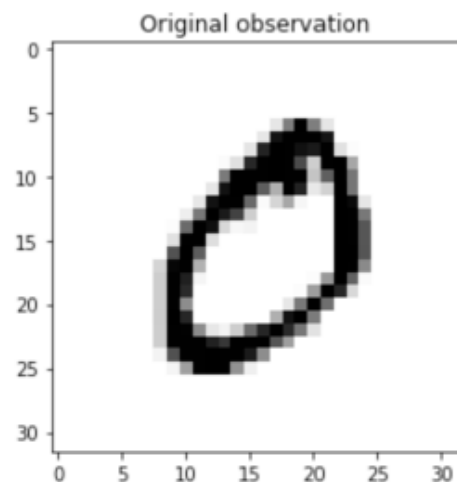


Practical case



Jeu de données MNIST [LeCun et al. 1998]





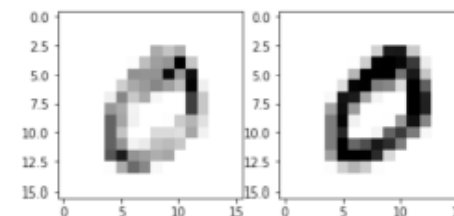
X_i

Convolutions avec

Filter 1:
 $\begin{bmatrix} 0.17653309 & 0.08818832 \\ -0.2810376 & 0.22499529 \end{bmatrix}$
 Bias 1:
 0.029923715

Filter 2:
 $\begin{bmatrix} 0.15315333 & 0.14285438 \\ 0.22419561 & 0.16171551 \end{bmatrix}$
 Bias 2:
 -0.085632496

puis ReLU, puis MaxPooling



Convolution et ReLU:

Couche

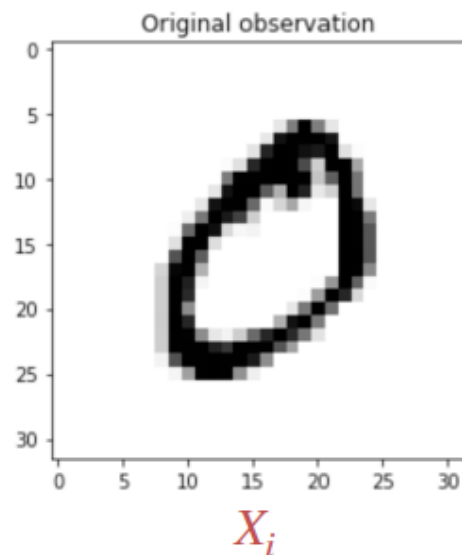
Canal

Biais du canal c
de la couche 2

Filtre du canal c
de la couche 2

$$I_i^{2,c}(p_x, p_y) = b_c^2 + \sum_{k=0}^1 \sum_{l=0}^1 X_i(p_x + k, p_y + l) w_c^2(k, l)$$

$$I_i^{2,c}(p_x, p_y) = \max(0, I_i^{2,c}(p_x, p_y))$$

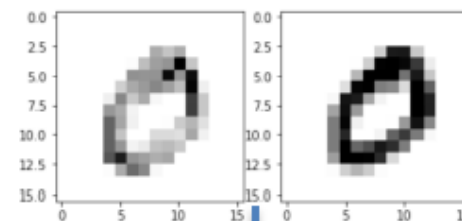


Convolutions avec

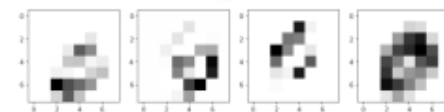
Filter 1:
 $\begin{bmatrix} 0.17653309 & 0.08818832 \\ -0.2810376 & 0.22499529 \end{bmatrix}$
 Bias 1:
 0.029923715

Filter 2:
 $\begin{bmatrix} 0.15315333 & 0.14285438 \\ 0.22419561 & 0.16171551 \end{bmatrix}$
 Bias 2:
 -0.085632496

puis ReLU, puis MaxPooling



Convolutions avec 2×4 filtres de convolution puis ReLU, puis Max-pooling



Convolutions avec 4×8 filtres de convolution puis ReLU, puis Max-pooling



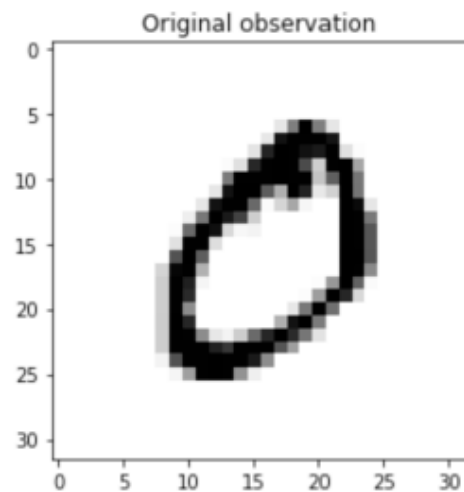
Convolution et ReLU:

Somme sur les canaux de la couche d'entrée

Filtre du canal c' de la couche 2 au canal c de la couche 3

$$I_i^{3,c}(p_x, p_y) = b_c^3 + \sum_{c'=1}^2 \left(\sum_{k=0}^1 \sum_{l=0}^1 I_i^{2,c'}(p_x + k, p_y + l) w_{c,c'}^3(k, l) \right)$$

$$I_i^{3,c}(p_x, p_y) = \max(0, I_i^{3,c}(p_x, p_y))$$



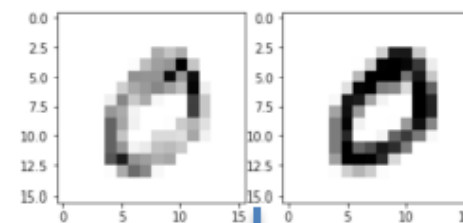
X_i

Convolutions avec

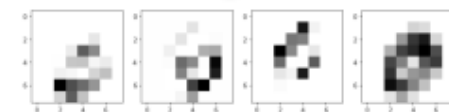
Filter 1:
 $\begin{bmatrix} 0.17653309 & 0.08818832 \\ -0.2810376 & 0.22499529 \end{bmatrix}$
 Bias 1:
 0.029923715

Filter 2:
 $\begin{bmatrix} 0.15315333 & 0.14285438 \\ 0.22419561 & 0.16171551 \end{bmatrix}$
 Bias 2:
 -0.085632496

puis ReLU, puis MaxPooling



Convolutions avec 2×4 filtres de convolution puis ReLU, puis Max-pooling



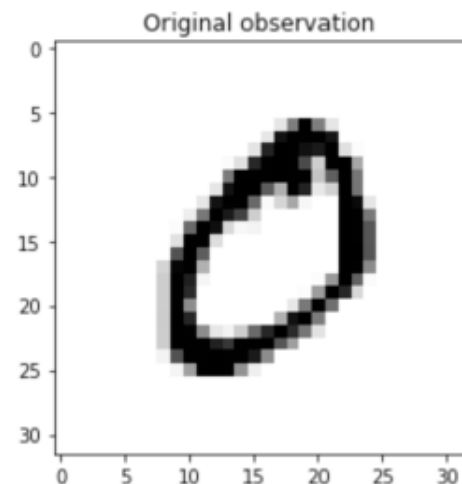
Convolutions avec 4×8 filtres de convolution puis ReLU, puis Max-pooling



Mise à plat des 8 images $I_i^{4,c}$ de taille 4×4 pixels

Vecteur V^5 de taille $8 \times 4 \times 4 = 128$

Espace des caractéristiques (ou *latent*)



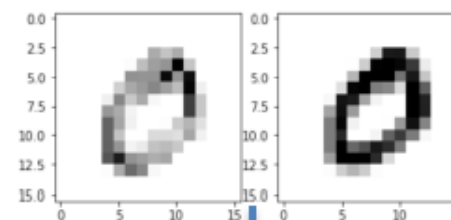
X_i

Convolutions avec

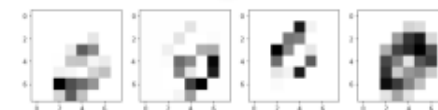
Filter 1:
 $\begin{bmatrix} 0.17653309 & 0.08818832 \\ -0.2810376 & 0.22499529 \end{bmatrix}$
 Bias 1:
 0.029923715

Filter 2:
 $\begin{bmatrix} 0.15315333 & 0.14285438 \\ 0.22419561 & 0.16171551 \end{bmatrix}$
 Bias 2:
 -0.085632496

puis ReLU, puis MaxPooling



Convolutions avec 2×4 filtres de convolution puis ReLU, puis Max-pooling



Convolutions avec 4×8 filtres de convolution puis ReLU, puis Max-pooling



Mise à plat des 8 images $I_i^{4,c}$ de taille 4×4 pixels

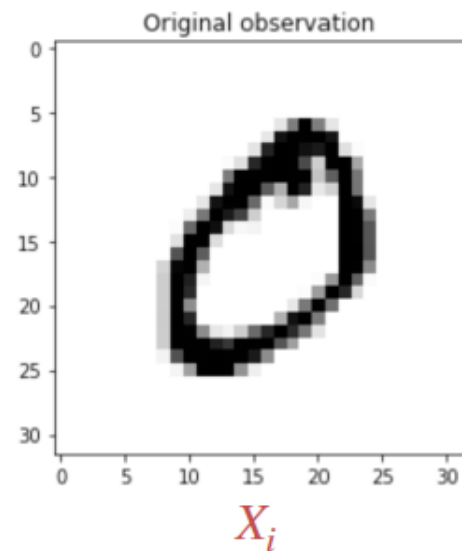
Chacun des 32 éléments est une somme pondérée des 128 précédents éléments :

$$V^6(i) = \text{ReLU} \left(\sum_{j=1}^{128} M^6(i, j) V^5(j) \right)$$

Vecteur V^6 de taille 32

Vecteur V^5 de taille $8 \times 4 \times 4 = 128$

Produit avec une matrice de taille 32×128 puis ReLU

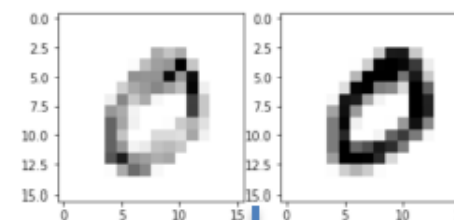


Convolutions avec

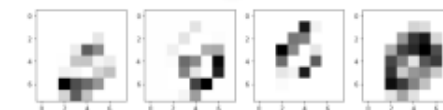
Filter 1:
 $\begin{bmatrix} 0.17653309 & 0.08818832 \\ -0.2810376 & 0.22499529 \end{bmatrix}$
 Bias 1:
 0.029923715

Filter 2:
 $\begin{bmatrix} 0.15315333 & 0.14285438 \\ 0.22419561 & 0.16171551 \end{bmatrix}$
 Bias 2:
 -0.085632496

puis ReLU, puis MaxPooling



Convolutions avec 2×4 filtres de convolution puis ReLU, puis Max-pooling



Convolutions avec 4×8 filtres de convolution puis ReLU, puis Max-pooling



Mise à plat des 8 images $I_i^{4,c}$ de taille 4×4 pixels

Vecteur V^5 de taille $8 \times 4 \times 4 = 128$

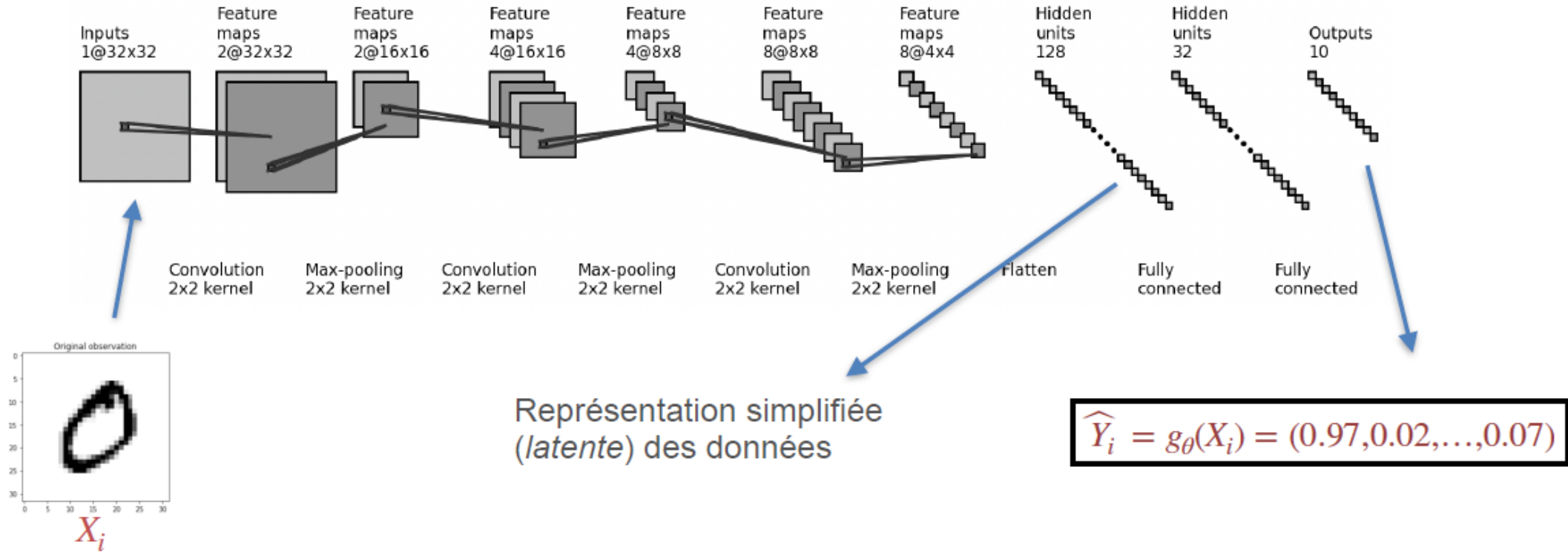
Produit avec une matrice de taille 32×128 puis ReLU

Vecteur V^6 de taille 32

$\hat{Y}_i = g_\theta(X_i) = (0.97, 0.02, \dots, 0.07)$

Produit avec une matrice 10×32 puis fonction Softmax :

$$\text{softmax}(s_k^7) = e^{s_k^7} / \left(\sum_{\kappa=1}^K e^{s_\kappa^7} \right)$$



Apprentissage du CNN : Optimisation des [filtres de convolution], [biais] et [matrices des dernières couches] afin qu'ils conduisent aux meilleurs prédictions possibles sur un jeu d'apprentissage.

- Les filtres ressortent des propriétés pertinentes dans l'image
- Les différents canaux mélangés peuvent créer des représentations avancées de l'information
- A l'aide des biais, les fonctions ReLU permettent de supprimer l'information inutile
- Le max-pooling sélectionne dans chaque région de l'image l'information la plus pertinente.